

Staatlich geprüfte/r Informationstechnische/r Assistent/in

Facharbeit zum Betriebspraktikum

Thema: Frontend Webdesign für ein Internes
Lagerverwaltungssystem

Name: Paul Dobrick

Klasse TAIF3

Beginn: Montag, 10. Februar 2025

Ende Freitag, 04. April 2025

Abgabedatum: Montag, 5. Mai 2025

Fachlehrer/in: Herr Maluck

Die vorgelegte Facharbeit wird mit der Note

bewertet.

Olsberg, den _____

(Datum)

(Unterschrift der betreuenden Lehrkraft)

Inhaltsverzeichnis

1	Vorstellung des Betriebes	1
2	Zusammenfassung.....	1
3	Beschreibung des betrieblichen Auftrags	4
4	Dokumentation	6
	4.1 Planung.....	6
	4.2 Beschreibung der Durchführung	7
	4.2.1 Verbindung zur Schnittstelle.....	8
	4.2.2 Benutzerdefinierte Rechte.....	10
	4.2.3 Tatsächliche Umsetzung in Codeform.....	11
	4.2.4 Der Login Algorithmus.....	14
	4.2.5 Dynamische Anpassung der Website an Bildschirmverhältnisse.....	21
	4.2.6 Laden der benötigten Daten über die GraphQL-Schnittstelle	24
	4.3 Dokumentation der Ergebnisse	32
	4.3.1 Funktionsumfang	32
	4.3.2 Technische Umsetzung.....	33
	4.4 Diskussion der Ergebnisse	33
5	Tagesberichte	34
6	Eigenständigkeitserklärung.....	42
7	Praktikumsbescheinigung	43
	Literaturverzeichnis.....	45

Abbildungsverzeichnis

Abbildung 1: Die Scanner der Firma «Zebra» sind in der Richter Elektronik GmbH bereits vertreten und können somit perspektivisch auch gut in das Lagerverwaltungssystem eingebunden werden. Sie basieren auf Android und stellen alle nötigen Funktionalitäten, sowie eine QR / Barcode-Scanner zur Verfügung.....	2
Abbildung 2: Links die bereits bestehende Funktionierende Weboberfläche und Rechts die neue Intuitivere UI (Die Leeren Kacheln könnten bei einer Erweiterung des Systems zur Erweiterung des Dashboards gefüllt werden)	2
Abbildung 3: Die Systemanbindung erfolgte über eine GraphQL-Schnittstelle. Diese verknüpft definierte Schemafelder mit entsprechenden Funktionen und ermöglicht dadurch die unkomplizierte Nutzung komplexer Funktionalitäten durch einfache, interaktive Dropdown-Menüs.	3
Abbildung 4: In diesem Bild sehen Sie den Workflow mit "Gitea". Zu jedem Projekt, welches hier Firmenintern veröffentlicht wird, können verschiedene Aufgaben auch "Issues" genannt Kategorisiert, Erstellt und Mitarbeitern hinzugefügt werden	4
Abbildung 5: Screenshot des Nightingale REST Clients. Die Antwort des Servers wird als JSON im unteren Bereich angezeigt.....	7

Abbildung 6: Zusehen ist die Login Seite der Stockpile Website. Das Tor zum Lagerverwaltungssystem. Wer die anderen Seiten betreten und nutzen möchte muss sich hier mit seinem Windows Nutzerprofil einloggen.	11
Abbildung 7: Sequenzdiagramm der Login-Funktion.....	21
Abbildung 8: Darstellung der Startseite, sowohl auf dem PC, als auch auf dem Handy	24
Abbildung 9: Aufbau eines Vue.js Projekts. Komponenten lassen sich schnell und flexible austauschen.....	25
Abbildung 10: Die Tabelle, welche ein Nutzer auf der Desktop Startseite zu sehen bekommt	29

Tabellenverzeichnis

Tabelle 1: Auflistung einiger HTTP-Methoden.....	9
Tabelle 2: Optionale Bereiche in einer API-Struktur	13
Tabelle 3: Zusammenfassung aller API-Strukturen	14

1 Vorstellung des Betriebes

Die **Richter Elektronik GmbH** ist ein mittelständisches Unternehmen mit Sitz in **Schmallenberg** im Sauerland. Gegründet wurde es zwischen 1978 und 1989 als **Ein-Mann-Betrieb** von **Theo Richter**. Schon in den Anfangsjahren wuchs das Unternehmen auf **15 Mitarbeiter** an, und die Produktionsfläche konnte von **75 auf 2.200 Quadratmeter** erweitert werden.

Als einer der **ersten Leiterplattenhersteller Deutschlands** setzte Richter Elektronik von Beginn an auf Qualität „**100% Made in Germany**“. Durch kontinuierliche Investitionen in **zukunftsweisende Technologien** konnte sich das Unternehmen langfristig als **schneller und zuverlässiger Partner** insbesondere für Forschungsinstitute etablieren.

Ein entscheidender Wettbewerbsvorteil des Unternehmens liegt in der **flexiblen und individuell angepassten Produktion**, die sich von standardisierten Massenproduktionen abhebt. Während große Hersteller oft günstiger produzieren können, bietet Richter Elektronik seinen Kunden **maßgeschneiderte Lösungen mit kurzen Lieferzeiten** – ein wesentlicher Faktor für Unternehmen und Forschungseinrichtungen, die auf **schnelle Prototypenentwicklung und hochwertige Spezialanfertigungen** angewiesen sind.

Heute beschäftigt Richter Elektronik rund **65 Mitarbeiter** und verfügt über eine hochmoderne Fertigung. Besonders in der **Produktion komplexer Multilayer-Leiterplatten** hat sich das Unternehmen einen Namen gemacht. Damit ist es ein herausragendes Beispiel dafür, dass **Innovation und technologische Entwicklung auch in einer ländlichen Region wie Schmallenberg möglich sind**.

2 Zusammenfassung

Das in dem 8-wöchigen Praktikum zu entwickelndem Projekt umfasste die Umsetzung einer benutzerfreundlichen und optisch ansprechenden Lagerverwaltungswebsite.

Diese sollte perspektivisch vermehrt auf mobilen Geräten (insbesondere den "Zebra"-Scannern) zum Einsatz kommen, weshalb ein besonderes Augenmerk auf die Gestaltung der Seite im Hinblick auf kleinere Auflösungen gelegt werden musste.



Abbildung 1: Die Scanner der Firma «Zebra» sind in der Richter Elektronik GmbH bereits vertreten und können somit perspektivisch auch gut in das Lagerverwaltungssystem eingebunden werden. Sie basieren auf Android und stellen alle nötigen Funktionalitäten, sowie eine QR / Barcode-Scanner zur Verfügung.

Mein Projekt baute auf dem bestehenden "Stockpile"-Konzept der IT-Abteilung auf. Die Aufgabe bestand darin, eine neue Frontendⁱ-Lösung zu entwickeln, die mit der bestehenden Systemarchitekturⁱⁱ kompatibel ist.

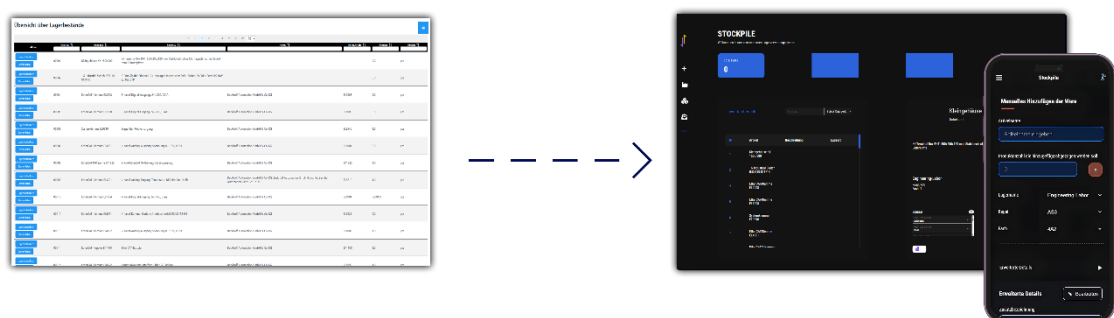


Abbildung 2: Links die bereits bestehende funktionierende Weboberfläche und Rechts die neue intuitive UI (Die Leeren Kacheln könnten bei einer Erweiterung des Systems zur Erweiterung des Dashboards gefüllt werden)

Die vorherige Benutzeroberfläche erfüllte nicht die gestalterischen Anforderungen und war besonders auf mobilen Geräten nur eingeschränkt bis garnicht nutzbar.

Ein Umbau der bereits bestehenden Website wäre nicht sinnvoll gewesen, da dies einiges mehr an Arbeit mit sich bringen würde, weshalb ich dazu angeleitet wurde, eine neue Website aufzusetzen. Diese konnte jedoch die Funktionsweise im Hintergrund (beispielsweise den tatsächlichen Speicherprozess von Daten wie Artikellagerplätzen) über eine von meinem Betreuer Herrn Christes parallel umgesetzte Anbindung nutzen. So war es meine Aufgabe die Eingaben eines Benutzers auslesen und zum passenden Zeitpunkt an die richtige Adresse zu senden. Alle im Hintergrund laufenden Prozesse wurden durch das bereits bestehende System abgewickelt.

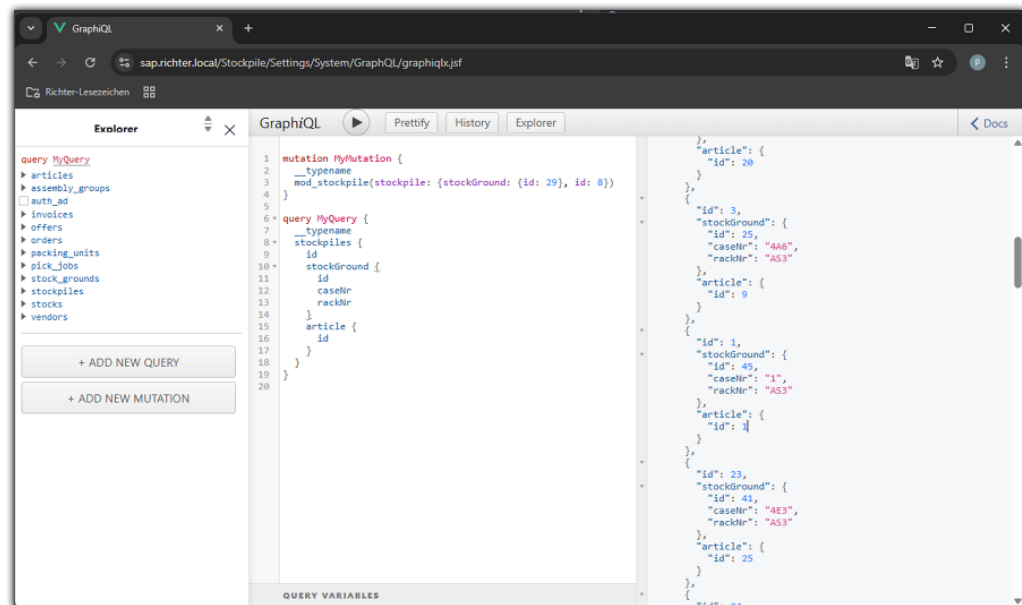


Abbildung 3: Die Systemanbindung erfolgte über eine GraphQL-Schnittstelle. Diese verknüpft definierte Schema-felder mit entsprechenden Funktionen und ermöglicht dadurch die unkomplizierte Nutzung komplexer Funktionalitäten durch einfache, interaktive Dropdown-Menüs.

Das "Stockpile"-Projekt wurde ins Leben gerufen, um Arbeitsabläufe effizienter zu gestalten, potentielle Lagerengpässe frühzeitig zu identifizieren sowie zeitnah geeignete Maßnahmen ergreifen zu können.

Diese sehr allgemeine Definition bot großen Interpretationsspielraum, weshalb in Absprache mit dem Betrieb anfangs konkrete Funktionen definiert wurden, die bis zum Ende des Praktikums funktionsfähig sein sollten, um das Projekt als erfolgreich abgeschlossen bezeichnen zu können.

Diese Funktionen lauteten wie folgt:

- Hinzufügen (Einlagern) von Bauteilen zur bestehenden Datenbankⁱⁱⁱ
- Entfernen (Entnahme) von Bauteilen aus der bestehenden Datenbank bzw. von Lagerplätzen
- Umlagern von Bauteilen in andere Fächer/Regale

- Dementsprechende Datenbankanpassung

Des Weiteren sollte die Website passwortgeschützt werden. Dies sollte mit den in der Firma für den Windows-Login angelegten Nutzerprofilen möglich sein.

3 Beschreibung des betrieblichen Auftrags

Aus Fachlicher Sicht bedeutet mein Projekt folgendes:

Das Projekt umfasste die Entwicklung einer Frontend-Lösung zur Anbindung an ein bestehendes Backend-System, das als Grundlage für die Lagerverwaltung "Stockpile" diente. Die zentrale Herausforderung bestand in der Erstellung einer responsiven Weboberfläche, die insbesondere für den Einsatz auf mobilen Endgeräten, speziell den "Zebra"-Scannern, optimiert werden sollte. **(Siehe Abb.1)**

Die Backend-Infrastruktur basierte auf Java und stellte ihre Funktionalitäten über eine GraphQL-Schnittstelle bereit. Diese Schnittstelle ermöglichte den strukturierten Datenaustausch zwischen dem Frontend und dem Backend-System. Die parallele Entwicklung der benötigten GraphQL-Endpunkte durch meinen Betreuer Robin Christes erforderte einen koordinierten Arbeitsablauf, der durch tägliche Absprachen und die Nutzung des Versionsverwaltungssystems Gitea^{iv} realisiert wurde. Gitea diente dabei nicht nur der Quellcodeverwaltung, sondern auch dem Projektmanagement durch die systematische Erfassung von Anforderungen und Problemen in Form von "Issues", den einzelnen Entwicklern zugewiesen werden konnten.

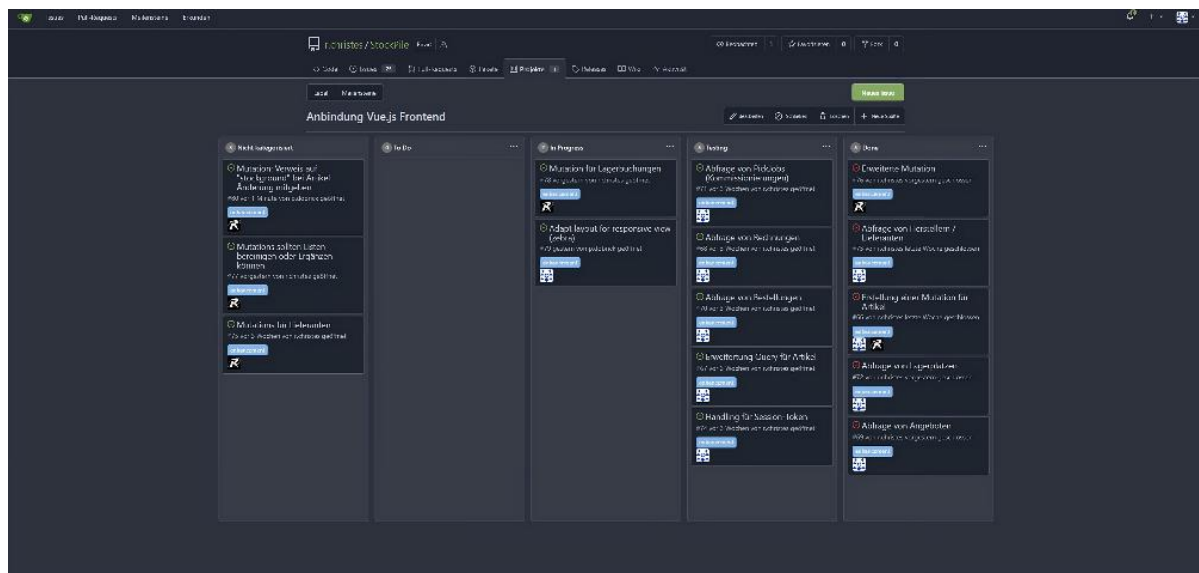


Abbildung 4: In diesem Bild sehen Sie den Workflow mit "Gitea". Zu jedem Projekt, welches hier Firmenintern veröffentlicht wird, können verschiedene Aufgaben auch "Issues" genannt Kategorisiert, Erstellt und Mitarbeitern hinzugefügt werden

Für die technische Umsetzung des Frontends wurde ein modernes Technologie-Stack vorgegeben, bestehend aus:

- **Vue.js:** Als progressive JavaScript-Framework ermöglichte Vue.js die Entwicklung einer komponentenbasierten Benutzeroberfläche, wodurch eine hohe Wiederverwendbarkeit und einfache Wartbarkeit gewährleistet wurde.
- **TypeScript:** Der Einsatz von TypeScript als typisierte Erweiterung von JavaScript erhöhte die Codequalität durch frühzeitige Fehlererkennung und verbesserte Entwicklungsunterstützung wie automatische Typüberprüfung und Code-Vervollständigung.
- **Bootstrap:** Das CSS-Framework Bootstrap diente als Basis für das responsive Design und stellte eine umfangreiche Bibliothek vorgefertigter UI-Komponenten bereit, die speziell für die Anforderungen der mobilen Nutzung angepasst wurden.

Diese Technologiekombination wurde bewusst gewählt, um eine hohe Wiederverwendbarkeit der Komponenten zu erreichen und einen modularen Aufbau zu ermöglichen, bei dem einzelne Elemente bei Bedarf schnell ausgetauscht oder angepasst werden können. Besonders im Hinblick auf zukünftige Erweiterungen bietet dieser Ansatz erhebliche Vorteile.

Die Frontend-Lösung wurde unter Berücksichtigung moderner Webentwicklungsstandards konzipiert, mit besonderem Fokus auf:

1. **Responsive Design:** Die Benutzeroberfläche passt sich dynamisch verschiedenen Bildschirmgrößen an, mit spezifischen Optimierungen für die Zebra-Scanner.
2. **Progressive Enhancement:** Je nach verfügbarer Bildschirmfläche werden unterschiedliche Funktionalitäten und Darstellungsformen angeboten, um die Benutzererfahrung zu optimieren.
3. **Modularität:** Die Implementierung erfolgte nach dem Komponentenprinzip, um eine hohe Wiederverwendbarkeit und einfache Erweiterbarkeit zu gewährleisten.

Der funktionale Anforderungskatalog für das Frontend umfasste folgende Kernsysteme:

- **Artikelmanagement:**
 - Erfassung und Speicherung vollständiger Artikeldatensätze mit Metadaten wie Bezeichnung, Hersteller, EAN-Code, Beschreibung, Bestandsparametern (Mindest-/Maximalbestand, Nachbestellmenge) und Haltbarkeitsinformationen
 - Modifikation bestehender Artikelattribute mit Persistenz im Backend-System
- **Lagerplatzverwaltung:**
 - Zuweisung von Artikeln zu spezifischen Lagerplätzen mit Mengenangabe
 - Entfernung von Artikeln aus definierten Lagerplätzen
 - Umlagerungsfunktion für den Transfer von Artikeln zwischen verschiedenen Lagerplätzen
 - Bestandsanpassung von Artikeln an bestimmten Lagerplätzen
- **Inventursystem:**
 - Erfassung und Aktualisierung des tatsächlichen Lagerbestands einzelner Artikel
 - Abgleich zwischen Soll- und Ist-Bestand
- **Authentifizierung:**

- Integration einer Zugriffskontrolle basierend auf den firmeninternen Windows-Benutzerkonten

Bei der technischen Umsetzung war es von entscheidender Bedeutung, dass die neu entwickelte Frontend-Lösung nahtlos mit der existierenden Backend-Infrastruktur kommunizieren konnte. Dies wurde durch die systematische Nutzung der parallel entwickelten GraphQL-Schnittstelle realisiert. Meine Aufgabe umfasste dabei die Implementierung der Client-seitigen GraphQL-Queries und -Mutations, um Benutzereingaben zu erfassen, zu validieren und zum richtigen Zeitpunkt an die entsprechenden Endpunkte zu übermitteln.

Das Projekt war von vornherein auf Erweiterbarkeit ausgelegt, wurde jedoch im Rahmen des achtwöchigen Praktikums auf die oben genannten Kernfunktionalitäten beschränkt, um eine vollständige und qualitativ hochwertige Implementierung zu gewährleisten.

4 Dokumentation

4.1 Planung

Die Planung meines Praktikumsprojekts startete bereits einige Wochen vor dem eigentlichen Praktikumsbeginn durch eine offene und konstruktive Kommunikation zwischen mir und dem Unternehmen. Bei der Projektfindung wurden meine persönlichen Interessen und Wünsche aktiv berücksichtigt, was zu einer hohen Motivation meinerseits führte.

Dieser frühe Austausch erwies sich als äußerst vorteilhaft, da wir die Kernaspekte des Projekts und die grundlegenden Anforderungen bereits im Vorfeld definieren konnten. Dadurch verkürzte sich die notwendige Planungsphase während des eigentlichen Praktikums erheblich. Auch grundlegende Fragen bezüglich der einzusetzenden Frameworks und Programmiersprachen wurden frühzeitig geklärt.

Das Unternehmen empfahl mir den bereits erwähnten Technologie-Stack ⁹mit Vue.js, TypeScript und Bootstrap. Diese rechtzeitige Festlegung war für mich besonders wertvoll, da ich so die Möglichkeit hatte, mich bereits vor Praktikumsbeginn mit diesen Technologien vertraut zu machen. Am ersten Praktikumstag waren mir die meisten zu nutzenden Technologien somit bereits vertraut.

Der erste Tag konnte somit effektiv genutzt werden um einige Rahmenfunktionen festzulegen, welche nach Abschluss des Praktikums fertig gestellt sein sollten. Diese wurden bereits in der vorherigen Auflistung genannt.

Des Weiteren fand ich mich in der Implementierung einzelner Funktionen immer wieder in einer Planungs- bzw. Umplanungsphase wieder. Teils schienen einige Funktionen nicht so implementiert werden können wie gedacht. In diesen Momenten bedarf es dann kurze Rücksprache mit meinem Betreuer oder ein kurzes zurücktreten von dem Problem und die Betrachtung als Ganzes.

Formale Planungsdokumente wie Struktogramme oder ähnliche Unterlagen wurden nicht verlangt oder erstellt.

4.2 Beschreibung der Durchführung

Den Auftakt meines Praktikumsprojekts bildete die Implementierung der Artikeldatenabfrage. Um meine Kenntnisse im Vue.js-Framework aufzufrischen, entwickelte ich zunächst ein kleines Demonstrationsprojekt. Dieser Wiedereinstieg verlief reibungslos, da ich bereits vor Praktikumsbeginn gezielte Vorbereitungen getroffen hatte. Besonders die vom Unternehmen empfohlenen Tutorials von dem YouTube Kanal „*NetNinja*“, die mir während der im Vorhinein stattfindenden Kommunikation nahegelegt wurden, erwiesen sich als wertvolle Ressource für meine Einarbeitung.

Um mich in die Anbindung an die GraphQL-Schnittstelle mithilfe von TypeScript einzuarbeiten machte ich mit den sogenannten REST-APIs^{vi} vertraut. Dazu nutzte ich vorerst den **Nightingale REST Client**. Dieses Tool wurde speziell dafür entwickelt, um REST-APIs und dementsprechend auch die GraphQL-Schnittstelle zu testen.

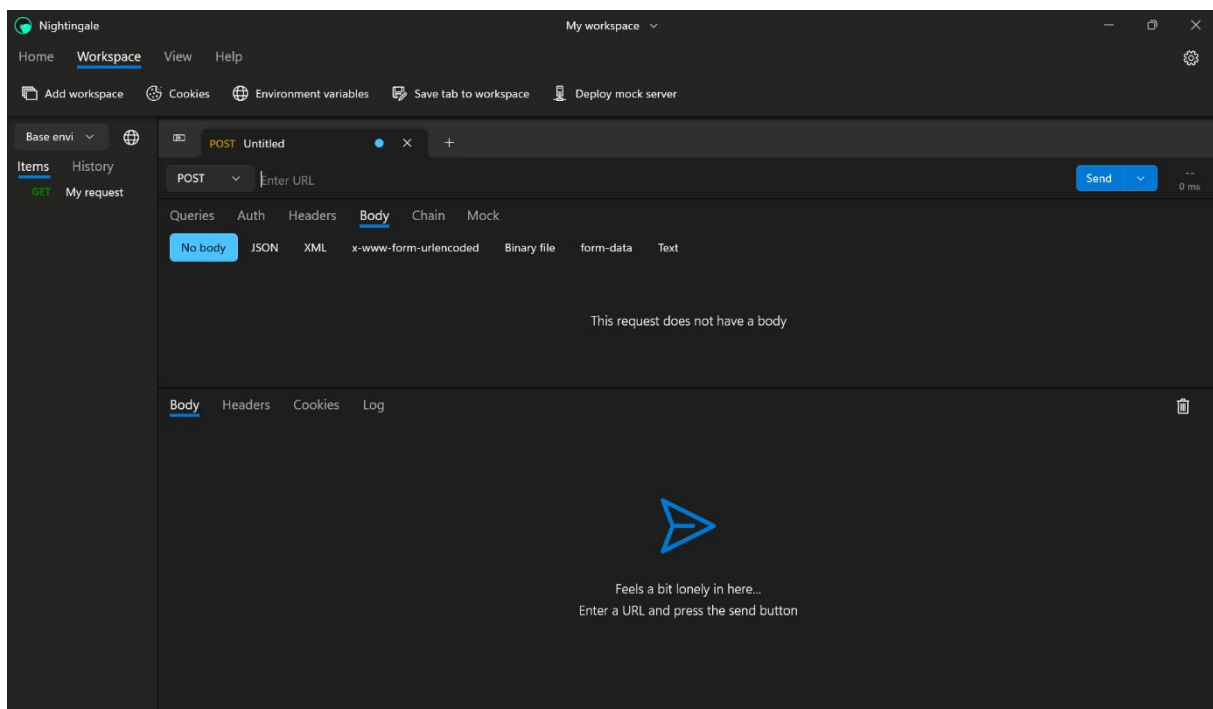


Abbildung 5: Screenshot des Nightingale REST Clients. Die Antwort des Servers wird als JSON im unteren Bereich angezeigt

Die Verbindung zur Schnittstelle wird mit folgenden Daten bewerkstelligt.:

4.2.1 Verbindung zur Schnittstelle

4.2.1.1 URL der Schnittstelle

Diese wird benötigt um zu definieren von welcher Adresse die Daten abgefragt werden sollen bzw. wohin Änderungen gesendet werden sollen. In meinem Fall handelt es sich dabei um eine Lokale Adresse meines Betriebes, welche die GraphQL-Schnittstelle hostet

4.2.1.2 API-Key

Dieser wird Standardmäßig nicht benötigt um eine Verbindung zur REST-API herzustellen, allerdings bei dem Großteil der produktiven APIs. Er fungiert als Schutz/Kontrollinstanz um Zugriffe von außerhalb zu kontrollieren, Benutzer zu identifizieren und dementsprechend Missbrauch zu verhindern. Nur wer den korrekten API-Key besitzt kann die Schnittstelle nutzen. So wurde dieser Schutzmechanismus auch in dem "Stockpile"-System genutzt. (Durch die bereits bestehende "Stockpile"-Demowebsite, welche die Funktion zur Erstellung/Verwaltung von API-Keys schon bereitstellte, konnten diese einfach angelegt und genutzt werden)

4.2.1.3 HTTP-Method

Es gibt mehrere HTTP-Methoden, welche in unterschiedlichen Anwendungsfällen zum Einsatz kommen. Sie werden unter anderem genutzt, um Daten wie Formulare an den Server zu übertragen oder Ressourcen und Informationen von einem Server abzurufen.

Folgend Liste ich einige (Nicht alle) Methoden um ein klareres Bild über die genaue Funktionsweise zu schaffen.:

Methode	Details
GET	Mit der GET-Methode lassen sich Daten laden, nicht aber übermitteln. Die erhaltenen Datensätze werden zudem sichtbar in der URL angezeigt
HEAD	Ähnelt der GET-Methode sehr, allerdings werden hierbei lediglich die Metadaten einer Ressource geladen (Infos wie: Content-Length, Content-Type, Last-Modified usw.) nicht aber der tatsächliche Inhalt
POST	Wird meist zur Übermittlung von Eingaben/Daten an einen Server genutzt. Kann aber auch Antworten von dem Server erhalten. Die gesendeten oder auch erhaltenen Daten werden hier allerdings nicht in der URL angezeigt.
PUT	Agiert ähnlich wie POST nur ist diese Methode idempotent bedeutet, dass auch bei mehrfachen Ausführungen lediglich Aktualisierungen vorgenommen werden, welche immer denselben Effekt haben und nicht plötzlich mehrere neue Elemente erstellt, so wie es bei POST vorkommen könnte.
DELETE	Löscht die angegebene Ressource

Tabelle 1: Auflistung einiger HTTP-Methoden

Die beiden meist genutzten sind dabei GET/POST und teils auch HEAD (um den korrekten Verbindungsaufbau zum Server zu testen).

In meinem "Stockpile"-Projekt nutzte ich ausschließlich die **POST-Methode** um die geladenen Daten nicht in der URL anzeigen zu lassen, da diese dann schnell extrem lang werden würde und hier keine Vorteile bietet.

4.2.2 Benutzerdefinierte Rechte

Um Benutzerdefinierte Rechte zuzuweisen und um einige Funktionen für vereinzelte Nutzer bereit zu stellen oder auch andere zu verbergen kann jedem Nutzer basierend auf Nutzernamen und Passwort ein eigener Session Token zugewiesen werden, welcher Nutzer kennzeichnet und worauf dementsprechend Regeln angewendet werden können. Diese Regeln werden im Backend festgelegt, müssen allerdings mithilfe des Frontend erfasst werden. Zudem müssen gestalterische Maßnahmen ergriffen werden, um blockierte Funktionen oder Extras zu verbergen/darzustellen.

Beispiel: Admin soll bestimmte Tabelle bearbeiten können - Standard Nutzer allerdings nur einsehen

Durch festlegen der dazu passenden Regel im Backend bleibt dem Standard Nutzer das senden der Abfrage versperrt (Das Bearbeitungsmenü kann aufgerufen werden, doch die Eingabe/Vermittlung an die Schnittstelle hat keine Wirkung) Um zu verhindern, dass der Standardnutzer dieser Bearbeitungsmodus überhaupt zu Gesicht bekommt, was suggeriert, dass dieser auch nutzbar ist, müssen Anpassungen basierend der Regeln im Frontend getroffen werden

Die Erfassung des Nutzers erfolgt im Fall meines Praktikumsprojekts über den:

1. Benutzername

&

2. Passwort

Um nicht jedem Mitarbeiter der Firma ein neues Nutzerprofil anlegen zu müssen, werden diese mithilfe der **LDAP-Authentifizierung** geladen. Dieses "*Lightweight Directory Access Protocol*" bietet die Verifizierung über beliebige Verzeichnisdienste. In unserem Fall wird dazu das **Microsoft Active Directory** genutzt.

Die Kontrolle über die Rechte wird über benutzerdefinierten Session-Tokens gewährleistet. Der API-Key, den ich bereits im oberen Abschnitt erläuterte, ist für jeden Zugriff auf die Seite identisch und muss daher bestehen bleiben. Allerdings wird dieser beim Login nach dem Abgleich von Benutzername und Passwort mit dem Active Directory (AD) durch einen **Session-Token ersetzt**.

Von diesen Session-Tokens können beliebig viele erstellt werden, welche je nach Konfiguration alle unterschiedlichen Rechtebereiche besitzen. Diese können dann den entsprechenden Nutzern zugewiesen werden, wodurch diese im Backend genau die Rechte erhalten, die für den jeweiligen Token festgelegt wurden.

In der Praxis hat es sich bewährt, separate Token-Profile für verschiedene Nutzergruppen anzulegen – beispielsweise für bestimmte Abteilungen, für Standardnutzer und für Administratoren. Dies ermöglicht eine flexible, aber trotzdem übersichtliche Verwaltung der Zugriffsrechte. Da die Funktion zur Festlegung und Anwendung der Regeln auf die Tokens noch nicht

implementiert war bezog ich diesen zwar in mein Programm ein, eine Wirkung hatte die Nutzung des Session-Tokens allerdings nicht und wurde auch bis zum Ende des Praktikums nicht umgesetzt, da hierdrauf kein besonderes Augenmerk lag.

In Zukunft könnte dies so umgesetzt werden: Der Server prüft bei einer Anfrage meiner Website den mitgesendeten Token im Header. Er gleicht die Token-Berechtigung mit der Anfrage ab. Bei fehlender Berechtigung sendet er einen Boolean-Wert zurück. Meine Website verarbeitet diesen Wert und blendet dann die entsprechende Tabelle oder Funktion aus.

4.2.3 Tatsächliche Umsetzung in Codeform

Die Umsetzung mithilfe des Nightingale Clients wurde allerdings nur für das Verständnis verwendet und musste schlussendlich als Code in TypeScript umgesetzt werden.

Dieser setzt sich fundamental aus der ausgelagerten Datei "articleService.ts", dem Pinia-Store^{vii} "selectedArticleStore.ts" und dem Login Vorgang in der "loginForm.vue" zusammen. Um die einzelnen Abfragen wiederverwendbarer zu gestalten kamen noch einige "load*.ts" dazu. Um eine flüssige und verständliche Struktur dieses Berichts bei zu behalten, orientiere ich mich an dem festgelegten Verlauf der Website. Diese kann nur über ein Erfolgreichen Login betreten werden.

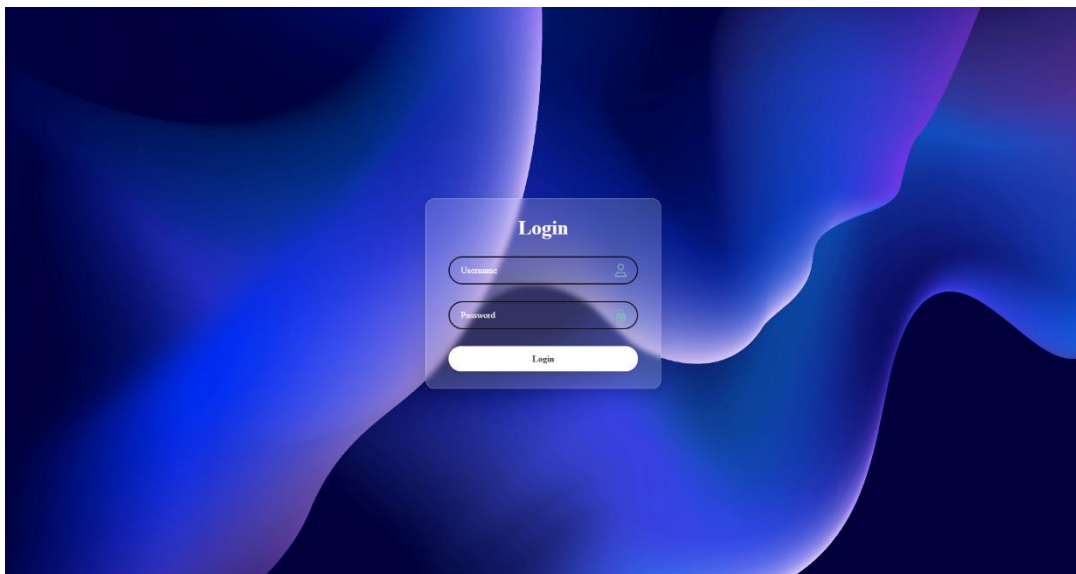


Abbildung 6: Zusehen ist die Login Seite der Stockpile Website. Das Tor zum Lagerverwaltungssystem. Wer die anderen Seiten betreten und nutzen möchte muss sich hier mit seinem Windows Nutzerprofil einloggen.

Die Login-Seite besteht aus einem Formular mit zwei Eingabefeldern (Benutzername & Passwort). Durch die Vue.js-Direktive `@submit.prevent="login"` werden die Eingaben an die Schnittstelle übermittelt, während das standardmäßige Verhalten eines Formulars blockiert wird. Dies verhindert das Neu laden der Website, was in unserem Fall unerwünscht wäre, da dabei die Antwort der Schnittstelle verloren ginge.

Der CSS-Teil ist zwar für die Darstellung zuständig, hat jedoch keinen Einfluss auf die Funktionalität und wird daher nicht näher erläutert. Wesentlich relevanter ist das TypeScript, welches

die eigentliche Funktionalität steuert. In meinem Projekt habe ich den TypeScript-Code größtenteils ausgelagert, allerdings befindet sich der Code für die Login-Komponente aufgrund anfänglicher Unklarheiten noch in derselben Datei. Da TypeScript in Verbindung mit Vue.js verwendet wird, existieren verschiedene Ansätze und Strukturen für die Erstellung von Komponenten.

4.2.3.1 Vue.js Strukturen

4.2.3.2 Options API

Verwendet `data()`, `methods`, `computed`, `watch` etc.

```
<script>
export default {
  // Reaktiver Zustand der Komponente
  data() {
    return {
      // Variablen für Formulare, Zähler, Zustandsdaten, etc.
    };
  },

  // Funktionen für Events oder interne Logik
  methods: {
    // Methoden zur Datenverarbeitung und UI-Interaktion
  }
};
</script>
```

Vorteile:

- Einfach zu lesen für Anfänger
- Klare Trennung von Optionen

Diese Struktur wird von Vue 3 noch immer Unterstützt, gilt allerdings als veraltet und wurde dementsprechend nicht von mir verwendet (Ausnahme ist die Login-Seite da hier für mich noch nicht klar war, welche Version genutzt werden sollte. Dies ließe sich allerdings ebenfalls noch umbauen)

4.2.3.3 Composition API

Verwendet `setup()` – ein spezieller Funktionsblock

```

<script>
import { ref } from 'vue';

export default {
  // setup() ist der zentrale Bereich für Logik und Zustand
  setup() {
    //Reaktive Werte (Zahlen, Strings, Booleans)

    // Rückgabe für das Template
    return {
      // Alle genutzten Werte und Funktionen
    };
  }
};
</script>

```

TypeScript

Sowohl der Options API Ansatz, als auch der Composition API Ansatz kann durch einige weitere Bereiche erweitert werden:

Bereich	Zweck
computed	Abgeleitete, gecachte Werte
watch	auf Änderungen reagieren
mounted	Lifecycle-Hook, nach Mounting
props	Daten von außen empfangen

Tabelle 2: Optionale Bereiche in einer API-Struktur

Seit neusten gibt es nun auch noch den:

4.2.3.4 Script Setup Syntax

Diese ist noch kompakter und wird für neue Projekte empfohlen. Allerdings bin ich darüber erst später gestolpert, weshalb dieser Ansatz in meinem Projekt nicht zum Einsatz kommt.

Die Struktur ähnelt der standartmäßigen JavaScript/TypeScript Struktur sehr und unterscheidet sich eigentlich nur in dem Öffnen Tag und dem Import von Vue:

```

<script setup> // Statt <script> wird <script setup> verwendet
import { ref } from 'vue'; // Hier erfolgt der Import von vue

/**CODE**/

</script>

```

TypeScript

4.2.3.5 Zusammenfassung

Ansatz	Name	Status	Vue Version
data(), methods	Options API	Klassisch	Vue 2 & 3
setup()	Composition API	Modern	Vue 3
<script setup>	Script Setup	Empfohlen	Vue 3.2+

Tabelle 3: Zusammenfassung aller API-Strukturen

4.2.4 Der Login Algorithmus

Um nun zurück zum Login zu kommen, schauen wir uns den Code im Detail an.

Der Login Algorithmus ist also wie folgt aufgebaut:

4.2.4.1 Das Login-Formular (HTML-Teil)

```

HTML
<template>
  <body>
    </img>
    <div class="wrapper">
      <form @submit.prevent="login">
        <h1>Login</h1>
        <div class="input-box">
          <input v-model="user.name" name="user" type="text" placeholder="Username">
          <i class="bx bx-user">
            <r-icon class="fb-user ri-sm2"/>
          </i>
        </div>

        <div class="input-box">
          <input v-model="user.pass" id="pass" name="pass" type="password"
placeholder="Password">
          <i class="bx bxs-lock-alt">
            <r-icon class="fb-lock ri-sm2"/>
          </i>
        </div>
        <button type="submit" class="btn">Login</button>
      </form>
    </div>
  </body>
</template>

```

In diesem Teil:

- Das Formular hat zwei Input-Felder für Benutzername und Passwort
- Beide Felder sind mit `v-model` an Datenproperties (user.name und user.pass) gebunden

- `@submit.prevent="login"` verhindert den Standard-Formular-Submit und ruft stattdessen die login()-Methode auf

4.2.4.2 Die Daten-Vorbereitung (script setup und data)

```
TypeScript
setup() {
  const router = useRouter();
  return { router };
},

data() {
  return {
    user: {
      name: "",
      pass: "",
      auth_ad: ""
    } as User,
    apiKey: "b25384a532dd3c3831acd71",
    apiUrl: "https://sap.richter.local/api/v1/graphql?"
  };
},

created() {
  // Beim Komponenten-Start: API-Key in SessionStorage speichern
  if (!sessionStorage.getItem("apiKey")) {
    sessionStorage.setItem("apiKey", this.apiKey);
  }
}
```

In diesem Teil:

- `setup()` initialisiert den Router für spätere Navigation
- `data()` definiert reaktive Daten:
 - Ein User-Objekt mit name, pass und auth_ad
 - Den API-Key und die API-URL
- `created()` speichert den API-Key im sessionStorage beim Laden der Komponente

4.2.4.3 Die Headers-Hilfsmethode

```
getHeaders(useAuthAd: boolean = false): Headers {
    const headers = new Headers({
        'Content-Type': 'application/json'
    });

    if (useAuthAd && this.user.auth_ad) {
        headers.append('auth_ad', this.user.auth_ad);
    } else {
        const sessionApiKey = sessionStorage.getItem("apiKey");
        if (sessionApiKey) {
            headers.append('X-api-key', sessionApiKey);
        }
    }

    return headers;
}
```

TypeScript

Diese Methode:

- Erstellt Headers für API-Anfragen
- Hat einen Parameter `useAuthAd`, der bestimmt, ob der auth_ad-Token verwendet wird
- Falls useAuthAd=true ist und ein Token existiert, wird dieser verwendet
- Sonst wird der API-Key aus dem sessionStorage verwendet

4.2.4.4 Die Login-Methode

```
async login(): Promise<void> {
  const query = `
    query MyQuery {
      auth_ad(password: "${this.user.pass}", username: "${this.user.name}")
    }
  `;

  try {
    const response = await fetch(this.apiUrl, {
      method: 'POST',
      headers: this.getHeaders(),
      body: JSON.stringify({query})
    });

    const data = await response.json();

    // Store auth_ad - Token in this.user.auth_ad
    this.user.auth_ad = data?.data?.auth_ad || "";

    if (this.user.auth_ad && this.user.auth_ad.trim() !== "") {
      // Speichern des auth_ad Tokens in sessionStorage
      sessionStorage.setItem("auth_ad", this.user.auth_ad);
      console.log("Anmelden erfolgreich");

      // Weiterleiten zur Startseite
      this.router.push('/home');
    } else {
      console.log("Anmeldung fehlgeschlagen");
      this.user.auth_ad = "";
    }
  } catch (error) {
    console.error('Fehler beim Login:', error);
    this.user.auth_ad = "";
  }
}
```

Hier passiert Folgendes in dieser Reihenfolge:

1. Eine GraphQL-Query wird erstellt, um Benutzername und Passwort zu authentifizieren
2. Ein POST-Request wird an den API-Endpunkt gesendet (mit Headers von `getHeaders()`)
3. Die Antwort wird als JSON verarbeitet
4. Der `auth_ad`-Token wird aus der Antwort extrahiert
5. Wenn ein gültiger Token erhalten wurde:
 - Der Token wird im `sessionStorage` gespeichert

- Der Benutzer wird zur Home-Seite weitergeleitet (router.push('/home'))
6. Falls kein Token erhalten wurde oder ein Fehler auftritt, wird der auth_ad-Wert geleert

4.2.4.5 getUserRights-Methode (auskommentiert)

```
async getUserRights(): Promise<void> {
    const rightsQuery = `...`;

    try {
        const response = await fetch(this.apiUrl, {
            method: 'POST',
            headers: this.getHeaders(true), // true für auth_ad Verwendung
            body: JSON.stringify({query: rightsQuery})
        });

        const data = await response.json();
        // Verarbeite die Nutzerrechte...
    } catch (error) {
        console.error('Fehler beim Setzen der Nutzerrechte:', error);
        this.user.auth_ad = "";
    }
}
```

Diese (aktuell deaktivierte) Methode:

- Würde nach erfolgreichem Login die Benutzerrechte abfragen
- Verwendet bereits den auth_ad-Token für die Anfrage (getHeaders(true))
- Könnte verwendet werden, um zusätzliche Benutzerinformationen nach dem Login zu laden

Ist momentan Auskommentiert, da noch keine Rechte/Richtlinien Serverseitig erstellt wurden.

4.2.4.6 Router-Protection

```
router.beforeEach((to, from, next) => {  
  // Check if page requires auth  
  const publicPages = ['/']; // Public pages  
  const authRequired = !publicPages.includes(to.path); // Auth necessary?  
  
  // Call auth_ad token from sessionStorage  
  const loggedIn = sessionStorage.getItem("auth_ad");  
  
  // If auth is required && Token is non existent  
  if (authRequired && !loggedIn) {  
    return next('/');  
  }  
  
  next();  
});
```

Dieser Teil (in index.ts):

- Stellt den Login Schutz von Routen bereit
- Prüft vor jeder Navigation, ob das Ziel eine geschützte Route ist
- Wenn die Route geschützt ist und kein auth_ad-Token existiert, Umleitung zum Login
- Sonst erlaubt er die Navigation

4.2.4.7 Pinia Auth Store (in selectedArticleStore.ts)

```
export const useAuthStore = defineStore('auth', {  
  state: () => ({  
    loggedIn: false,  
    authToken: sessionStorage.getItem('auth_ad') || null, // Token speichern  
  }),  
  actions: {  
    setAuthToken(token: string) {  
      this.authToken = token;  
      this.loggedIn = true;  
      sessionStorage.setItem('auth_ad', token); // Token im sessionStorage  
speichern  
    },  
    logout() {  
      this.authToken = null;  
      this.loggedIn = false;  
      sessionStorage.removeItem('auth_ad'); // Token löschen  
    },  
  },  
  getters: {  
    isLoggedIn: (state) => state.loggedIn,  
    getAuthToken: (state) => state.authToken,  
  },  
});
```

Dieser Store:

- Speichert und verwaltet den Authentifizierungsstatus
- Stellt Methoden zum Setzen und Löschen des Tokens bereit
- Bietet Getters für den Login-Status und Token

WICHTIG: Wie bereits erwähnt nutzt ausschließlich der Login Vorgang die veraltete Options API Struktur!

Um den Login Vorgang noch einmal besser darzustellen folgt hier ein Sequenzdiagramm zum Ablauf:

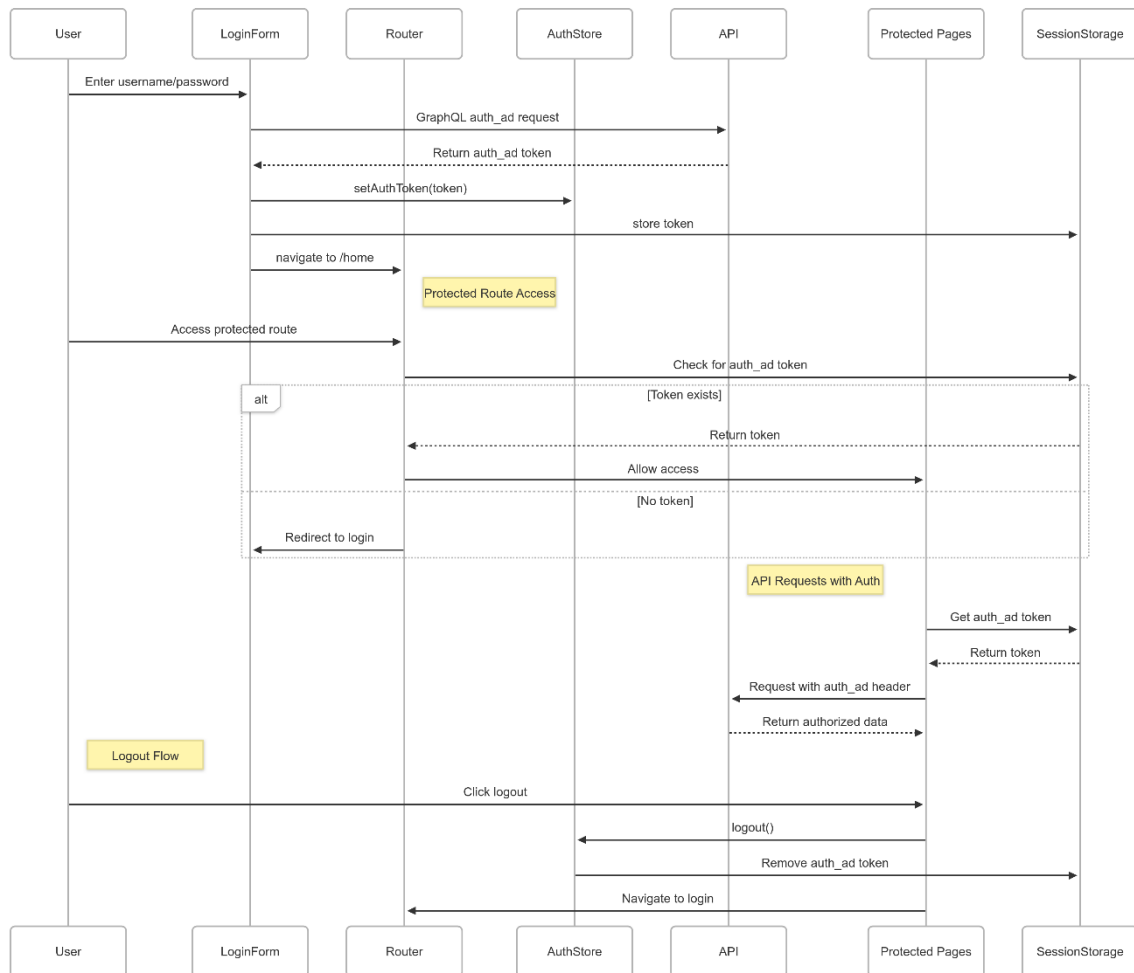


Abbildung 7: Sequenzdiagramm der Login-Funktion

Damit wäre der Login Vorgang beendet und der Nutzer findet sich auf einer seinem Gerät angepassten Startseite (Pfad: `/home`) wieder. Dazu wurden 2 Seiten erstellt, welche Abhängig der Bildschirmbreite des Aufrufenden Geräts, bezogen werden. Dabei wird bei Geräten von einer Breite unter 800px (Mobile Geräte) und einer Breite von über 800px (Tablet/Laptop/PC usw.) unterschieden.

4.2.5 Dynamische Anpassung der Website an Bildschirmverhältnisse

4.2.5.1 Konzeption der Gerätetyp-Erkennung

Für die Implementierung eines responsiven Designs war es zunächst notwendig, ein System zu entwickeln, das zwischen verschiedenen Gerätetypen unterscheiden kann. Die Lösung basiert auf dem Vue.js-Framework und nutzt reaktive Eigenschaften, um die Bildschirmbreite zu überwachen und entsprechend unterschiedliche Komponenten anzuzeigen.

Folgende Breakpoints wurden definiert:

```
const windowHeight = ref(window.innerWidth);
const isMobileView = computed(() => windowHeight.value <= 800);
const isTabletView = computed(() => windowHeight.value > 800 && windowHeight.value <= 1280);
const isDesktopView = computed(() => windowHeight.value > 1280);
```

Abbildung 6: Hier werden Breakpoints definiert, zwischen welchen Bildschirmgrößen Unterschieden werden soll. Im späteren Verlauf wird allerdings nur zwischen Desktop und Mobile unterschieden, da 4 Seitenversionen zu viel Arbeit mit sich ziehen würden

Diese computed properties ermöglichen eine dynamische Unterscheidung zwischen:

- Mobilien Geräten (bis 800px Bildschirmbreite)
- Tablets (zwischen 801px und 1280px)
- Desktop-Geräten (über 1280px)

4.2.5.2 Implementierung des Event-Listeners

Um auf Änderungen der Fenstergröße (z.B. bei Drehung des Geräts oder Größenänderung des Browsers) reagieren zu können, wurde ein Event-Listener für das 'resize'-Event implementiert:

```
const handleResize = () => {
  if (resizeTimeout) clearTimeout(resizeTimeout);
  resizeTimeout = setTimeout(() => {
    windowHeight.value = window.innerWidth;
    // Weitere berechnungen...
  }, 150);
};

onMounted(() => {
  window.addEventListener('resize', handleResize);
  handleResize(); // Initialer Check
});
```

Besonders wichtig war hier die Implementierung eines Debounce-Mechanismus^{viii} mittels setTimeout, um die Performance zu optimieren und zu verhindern, dass bei kontinuierlichen Größenänderungen zu viele Neuberechnungen ausgeführt werden.

4.2.5.3 Bedingtes Rendering der Komponenten

Das Herzstück der responsiven Implementierung bildet das bedingte Rendering im Template. Dabei werden vollständig unterschiedliche Layouts für mobile und nicht-mobile Geräte definiert:

```

<!-- Desktop und Tablet Layout -->
<template v-if="!isMobileView">
  <div class="header">
    <Header/>
  </div>
  <!-- Weitere Desktop/Tablet-Komponenten -->
</template>

<!-- Mobiles Layout -->
<template v-else>
  <div class="mobile-header">
    <mobileHeader/>
  </div>
  <!-- Weitere mobile Komponenten -->
</template>

```

HTML

Innerhalb des nicht-mobilen Layouts erfolgen weitere Unterscheidungen zwischen Desktop- und Tablet-Ansicht:

```

<div class="component2-top" v-if="isDesktopView || isTabletView">
  <!-- Komponente für Desktop und Tablet -->
</div>
<div class="component3-top" v-if="isDesktopView">
  <!-- Komponente nur für Desktop -->
</div>

```

HTML

4.2.5.4 Geräte-spezifische Komponenten

Ein wichtiger Aspekt der Implementierung war die Entwicklung spezialisierter Komponenten für verschiedene Gerätetypen:

Desktop-Komponenten:

- Header mit vollständiger Navigation
- Detaillierte Datenanalyse-Ansichten
- Umfangreiche Tabellenansichten der Lagerbestände

Mobile Komponenten:

- Kompakter Header
- Burger-Menü für die Navigation
- Vereinfachte Eingabemasken für Bestandsänderungen

Durch diesen Ansatz konnte sichergestellt werden, dass die Benutzeroberfläche nicht nur auf unterschiedliche Bildschirmgrößen angepasst, sondern tatsächlich für den jeweiligen Anwendungsfall optimiert ist.

4.2.6 Laden der benötigten Daten über die GraphQL-Schnittstelle

Nachdem der Nutzer nun erfolgreich eingeloggt ist und die Website sich an die Bildschirmgröße angepasst hat, werden die gewünschten Daten geladen, welche für die Aktuelle Übersicht benötigt werden.

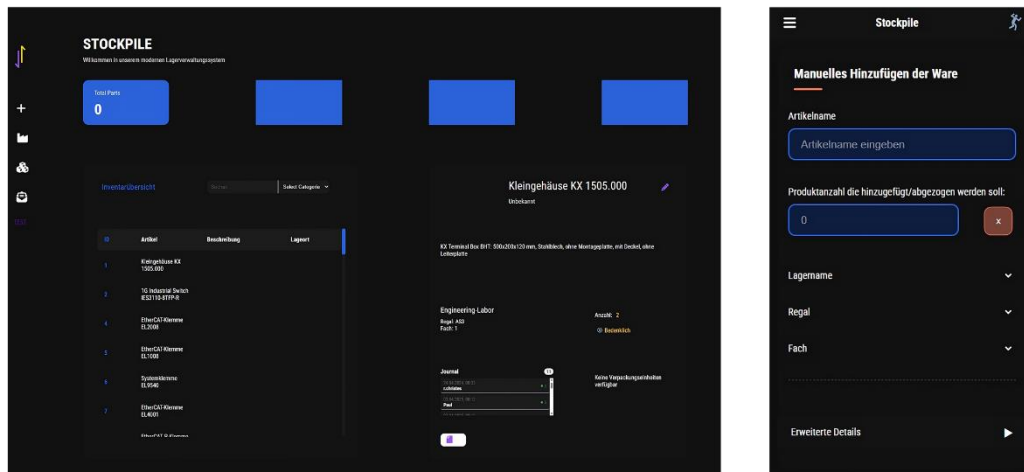


Abbildung 8: Darstellung der Startseite, sowohl auf dem PC, als auch auf dem Handy

Auf der /home Übersicht sind dies sowohl auf der Desktop Ansicht, als auch auf der Mobilen Ansicht die Artikel. Dabei ist es wichtig, dass nicht gleich alle Artikeldaten geladen werden sollten. Bei näherer Betrachtung der Seite kristallisiert sich schnell heraus, dass nur die Artikelnamen mit dessen ID geladen werden müssen. Weitere Produktspezifische Details müssen erst bei der Auswahl eines Artikels geladen werden.

Dieser Aufbau führt zu kürzeren Ladezeiten, da nur die nötigsten Daten geladen werden müssen. Anders würden bspw. auch gleich die Bilder aller Produkte geladen werden, bevor die Website erste Ergebnisse anzeigt. Diese führte bereits bei einer so kleinen Menge an Demoartikeln, wie sie bisher existiert, schon zu Ladezeiten von bis zu 5 Sekunden.

4.2.6.1 Architektur des Datenladesystems

Für die Implementierung des Artikelladesystems wurde ein mehrschichtiger Ansatz gewählt, der eine klare Trennung zwischen der Datenabfrage, der Datenverwaltung und der Darstellung ermöglicht. Die Hauptkomponenten dieses Systems sind:

1. **ArticleService**: Zuständig für die direkte Kommunikation mit der API
2. **useArticles Composable**: Kapselt die Logik und den Zustand der Artikeldaten
3. **fetchAPI.vue**: Verantwortlich für die visuelle Darstellung der Daten in Tabellenform

Diese Architektur folgt dem Prinzip der Separation of Concerns^{ix} und ermöglicht eine bessere Wartbarkeit sowie Testbarkeit des Systems. Dieses Prinzip wird auch bei allen weiteren Abfragen und Einträgen angewendet. Generalisiert bedeutet dies:

1. **ArticleService**: Beinhaltet die "fetch"-Requests/Mutations (Abfragen/Einträge)
2. **Composable**: Kapselt die Logik und den Zustand für eine bessere Wiederverwendbarkeit
3. **Vue.js Komponente**: Sorgt für die gezielte Darstellung EINER funktionalen Einheit
4. **Vue.js View**: Fasst mehrere Komponenten zu einer vollständigen Website zusammen

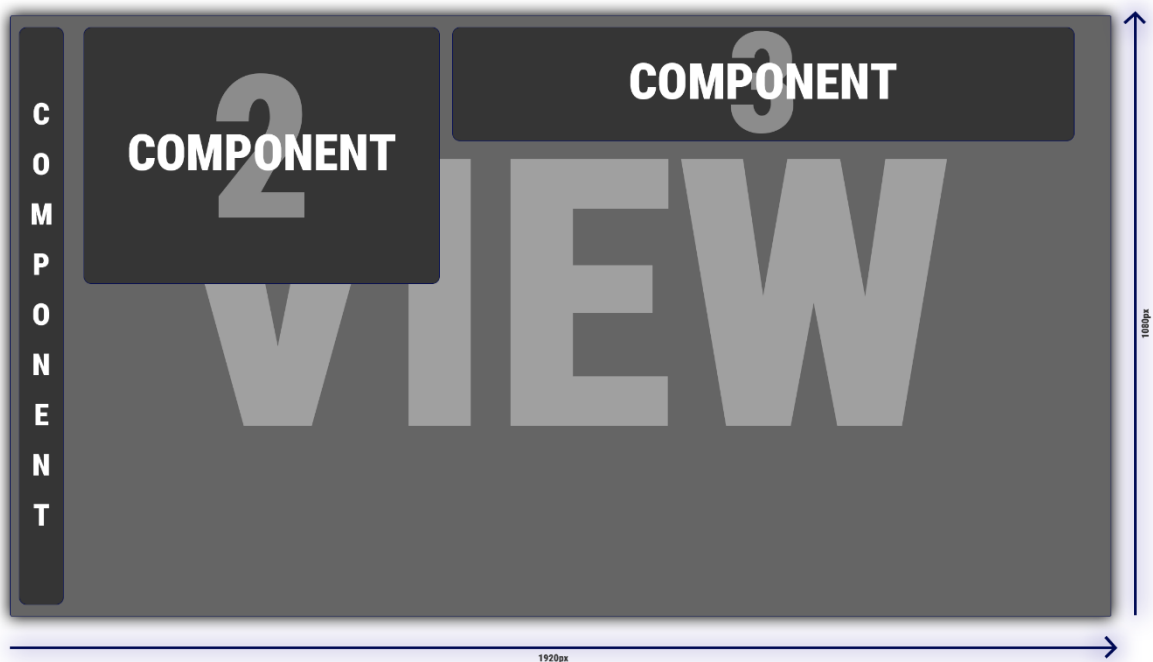


Abbildung 9: Aufbau eines Vue.js Projekts. Komponenten lassen sich schnell und flexible austauschen

4.2.6.2 Initialisierung des Datenladevorgangs

Die Initialisierung des Ladevorgangs erfolgt beim Mounting der Komponente über einen asynchronen Aufruf^{xi}:

```
onMounted(async () => {
  isLoading.value = true;
  await loadArticles();
  isLoading.value = false;
});
```

TypeScript

Dieser Ansatz stellt sicher, dass:

- Die Daten unmittelbar nach dem Rendering der Komponente geladen werden
- Ein Ladestatus während des Ladevorgangs angezeigt werden kann
- Fehler im Ladevorgang ordnungsgemäß behandelt werden können

4.2.6.3 Das useArticles Composable

Die Initialisierung ruft dabei die loadArticles()-Funktion im dazu gehörigen Composable auf. Diese ist wie folgt aufgebaut:

```
//useArticles.ts TypeScript

// Imports der API
import { ref, reactive } from 'vue';
import { Article } from '@services/articleService';
import { ArticleService } from '@services/articleService';

export function useArticles() {

  // Deklaration von Variablen zur Erfassung von Artikel Anzahl, Status und Fehlern
  const numberOfArticles = ref<number>(0);
  const isLoading = ref<boolean>(false);
  const error = ref<string | null>(null);

  // Aufruf und erstellen der in ArticleService erstellten Article-Strukturen
  const articles = ref<any[]>([]);
  const articleService = new ArticleService();

  // Funktion zum Starten des Fetch-Vorgangs in der ArticleService-Datei
  // Wird später über die Komponente aufgerufen
  const loadArticles = async () => {
    articles.value = await articleService.fetchArticles();
  }

  // Rückgabe von den erfassten Daten / existierenden Funktionen
  return {
    articles,
    loadArticles,
    numberOfArticles,
    isLoading,
    error
  };
}
```

In dieser ist die tatsächliche Abfrage NICHT enthalten. Einzig alleine wird hiermit der Aufruf dieser über die erstellte Funktion Namens *loadArticles* erleichtert und einige weitere Status Informationen mitgegeben. Die tatsächliche Abfrage der Artikeldaten geschieht in der folgenden Datei:

4.2.6.4 ArticleService.ts

Die articleService.ts stellt das Herzstück der Kommunikation mit der GraphQL-Schnittstelle dar. Diese Datei stellt alle auf der Website genutzten Abfragen bzw. Schreiboperationen bereit.

Im oberen Bereich werden dazu die Interfaces erstellt, welche in der späteren async Programmierung verwendet wird. Ein Interface ist dabei ein Art Bauplan für Objekte, der definiert, **welche Eigenschaften und Typen ein Objekt haben muss** – aber ohne konkrete Implementierung.

Die garantiert:

1. **Typsicherheit:** Es legt genau fest, welche Eigenschaften und Typen eine Abfrage im späteren Verlauf haben soll. Das hilft, Fehler bereits während der Entwicklung zu erkennen.
2. **Code-Vervollständigung:** Die Entwicklungsumgebung (IDE) kann dadurch automatisch Vorschläge machen, welche Eigenschaften verfügbar sind, wenn später eine Abfrage mit dem erstellten Interface durchgeführt wird.
3. **Dokumentation:** Interfaces dokumentieren implizit die Datenstruktur, was besonders wichtig ist, wenn mehrere Entwickler am Projekt arbeiten.

Besonders bei dem senden von Daten an die Schnittstelle mithilfe einer Mutation, spielt die Nutzung solcher Interfaces eine sehr große Rolle. Dabei können Eingaben des Nutzers in der Struktur des Interfaces gespeichert werden, welche wiederum in der Mutation die Werte einpflegt.

Nach der Erstellung der Interfaces folgt nun der Generelle Teil der ArticleService-Klasse. Diese gilt für alle Anfragen/Schreiboperationen und stellt einige Allgemeine Funktionen und Variablen bereit:

```
export class ArticleService {  
    private apiKey: string = "5cd12637dasd128d8ffoa32uf"; // Beispiel apiKey  
    private apiUrl: string = "https://sap.richter.local/api/v1/graphql?";  
    private isFetching: boolean = false; // Läuft gerade eine Anfrage?  
  
    // Debugging Funktion - Prüft auf auftretende GraphQL Fehler  
    private logGraphQLErrors(data: any) {  
        if (data?.errors && Array.isArray(data.errors)) {  
            console.error('GraphQL-Fehler:', data.errors);  
            return true;  
        }  
        return false;  
    }  
}
```

In diesem Fall gibt es noch 2 weitere Variablen, welche auf Daten im Cache prüfen. Diese lauten wie folgt:

```
private cachedArticles: Article[] | null = null;
private lastFetchTime: number = 0;
```

TypeScript

Diese sind allerdings nicht für alle Anfragen Allgemeingültig und sollten bei Überarbeitung des Projekts anders eingebaut werden. In diesem Fall können beispielsweise nur Daten des Datentyps `Article[]` im Cache gespeichert werden.

Die Klasse definiert zunächst grundlegende Eigenschaften:

- `apiKey`: Authentifizierungsschlüssel für die API-Zugriffe
- `apiUrl`: Endpunkt der GraphQL-Schnittstelle
- `cachedArticles`: Zwischenspeicher für bereits abgefragte Daten
- `lastFetchTime`: Zeitstempel der letzten Datenabfrage
- `isFetching`: Status-Flag zur Kontrolle paralleler Anfragen

4.2.6.5 Cache-Mechanismus zur Performanceoptimierung

Ein zentrales Element der `ArticleService`-Klasse ist der implementierte Cache-Mechanismus:

```
async fetchArticles(): Promise<Article[]> {
  // If fetch is in progress : wait
  if (this.isFetching) {
    console.log('Bereits laufende Anfrage wird abgewartet...');
    // Warten auf Abschluss der aktuellen Anfrage
    await new Promise(resolve => setTimeout(resolve, 100));
    return this.fetchArticles();
  }
  // Does a cache from within 30 sec exist??
  const currentTime = Date.now();
  if (this.cachedArticles && this.cachedArticles.length > 0 &&
    currentTime - this.lastFetchTime < 30000) {
    console.log('Verwende gecachte Artikel:', this.cachedArticles.length);
    return this.cachedArticles;
  }
}
```

TypeScript

Dieser Mechanismus verbessert die Anwendungsperformance durch:

1. **Vermeidung redundanter Anfragen**: Parallele Anfragen werden verhindert
2. **Zeitbasiertes Caching**: Daten werden für 30 Sekunden zwischengespeichert
3. **Reduzierung der Serverlast**: Unnötige Serveranfragen werden minimiert

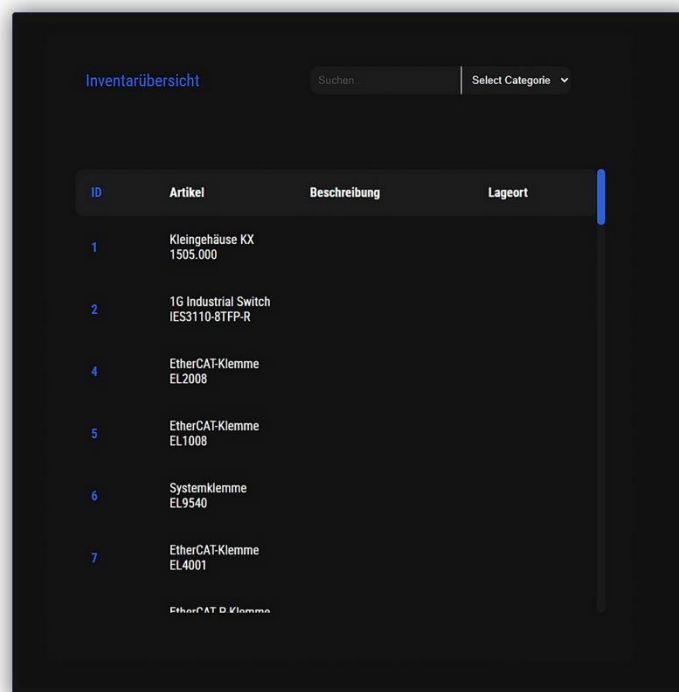
4.2.6.6 Minimale GraphQL-Abfrage

Die Datenabfrage erfolgt über eine präzise formulierte GraphQL-Query:

```
const query = `
  query {
    articles {
      desc1
      id
    }
  }
`;
```

TypeScript

Wie bereits erwähnt werden hier nur die Minimaldaten abgefragt, welche tatsächlich für die Ansicht gebraucht werden. Da auf der Startseite bei Aufruf erst einmal nur eine Tabelle erscheinen soll, in welcher Artikelnamen und dazugehörige IDs angezeigt werden, werden hier auch nur diese Informationen geladen.



The screenshot shows a web application interface with a dark theme. At the top, there is a header with the text 'Inventarübersicht' on the left, a search input field labeled 'Suchen' in the center, and a dropdown menu labeled 'Select Kategorie' on the right. Below the header is a table with four columns: 'ID', 'Artikel', 'Beschreibung', and 'Lageort'. The table contains seven rows of data. The first row has ID 1, Artikel 'Kleingehäuse KX', Beschreibung '1505.000', and Lageort. The second row has ID 2, Artikel '1G Industrial Switch', Beschreibung 'IES3110-8TFF-R', and Lageort. The third row has ID 4, Artikel 'EtherCAT-Klemme', Beschreibung 'EL2008', and Lageort. The fourth row has ID 5, Artikel 'EtherCAT-Klemme', Beschreibung 'EL1008', and Lageort. The fifth row has ID 6, Artikel 'Systemklemme', Beschreibung 'EL9540', and Lageort. The sixth row has ID 7, Artikel 'EtherCAT-Klemme', Beschreibung 'EL4001', and Lageort. The seventh row is partially visible and shows 'EtherCAT-Klemme'.

ID	Artikel	Beschreibung	Lageort
1	Kleingehäuse KX	1505.000	
2	1G Industrial Switch	IES3110-8TFF-R	
4	EtherCAT-Klemme	EL2008	
5	EtherCAT-Klemme	EL1008	
6	Systemklemme	EL9540	
7	EtherCAT-Klemme	EL4001	
	EtherCAT-Klemme		

Abbildung 10: Die Tabelle, welche ein Nutzer auf der Desktop Startseite zu sehen bekommt

Alle weiteren Produktspezifischen Daten müssen erst bei Klick auf dem jeweiligen Produkt geladen werden. Die Abfrage sieht allerdings ähnlich aus.

Hier ein Beispiel wie die Artikelspezifische Abfrage aussehen könnte (Abhängig der Daten, welche geladen werden sollen, kann diese natürlich etwas anders aussehen)

```

const query = `
  query {
    articles(id: [${index}]) {
      stockpiles {
        stockGround{
          id
          rackNr
          stock {
            id
            name
          }
        }
        caseNr
      }
    }
  }
  id
  stock
}
`

```

Die Variable "index" beinhaltet hier die ausgewählte Artikel ID, welche dafür sorgt, dass nur die einem bestimmten Produkt mit dieser ID zugehörigen Infos (in diesem Fall stockpiles-Details und die Artikelanzahl) geladen werden sollen.

Des Weiteren folgt der Klasse noch eine:

4.2.6.7 Robuste Fehlerbehandlung

```

if (!response.ok) {
  const errorText = await response.text();
  console.error(`HTTP-Fehler! Status: ${response.status}, Text: ${errorText}`);
  // IF HTTP-Request fails use cache
  if (this.cachedArticles && this.cachedArticles.length > 0) {
    console.warn('Verwende Cache nach HTTP-Fehler');
    return this.cachedArticles;
  }
  throw new Error(`HTTP-Fehler! Status: ${response.status}`);
}

```

Diese Fehlerbehandlung umfasst:

1. **Detaillierte Fehlerprotokolle:** HTTP-Status und Fehlermeldungen werden erfasst
2. **Automatischer Fallback:** Bei Fehlern wird auf den Cache zurückgegriffen
3. **Transparente Fehlermeldungen:** Präzise Fehlertypen werden weitergegeben

4.2.6.8 Hilfsfunktionen für die Entwicklung

Zur Unterstützung der Entwicklung enthält die Klasse eine Debug-Funktionalität:

```
private logGraphQLErrors(data: any) {  
    if (data?.errors && Array.isArray(data.errors)) {  
        console.error('GraphQL-Fehler:', data.errors);  
        return true;  
    }  
    return false;  
}
```

TypeScript

Diese Funktion erleichtert die Identifikation von GraphQL-spezifischen Fehlern während der Entwicklung und beim Produktionseinsatz.

```
const response = await fetch(this.apiUrl, {  
    method: 'POST',  
    headers: {  
        'Content-Type': 'application/json',  
        'X-api-key': this.apiKey,  
    },  
    body: JSON.stringify({query})  
});
```

TypeScript

Dieser Beispielabfrage folgen alle weiteren Abfragen. Auch das Hinzufügen von Artikeln erfolgt über dasselbe Prinzip. Der Unterschied besteht nur aus dem etwas abgewandelten Fetch-Aufbau. Statt der „const query“_ muss eine const Mutation gesendet werden. Diese folgt folgendem Beispiel:

```
const mutation = `
  mutation MyMutation {
    mod_articles(
      articles: {
        desc1: ${JSON.stringify(articleData.desc1)},
        desc2: ${JSON.stringify(articleData.desc2)},
        longDescription: ${JSON.stringify(articleData.longDescription)},
        ean: ${JSON.stringify(articleData.ean)},
        minStock: ${articleData.minStock},
        maxStock: ${articleData.maxStock},
        reorderStock: ${articleData.reorderStock},
        durabilityDays: ${articleData.durabilityDays},
        articleImages: [
          {
            title: ${JSON.stringify(articleData.articleImages?.title)},
            dataAsBase64: ${JSON.stringify(articleData.articleImages?.dataAsBase64)},
          }
        ],
        manufacturers: {
          manufacturer: {
            id: ${articleData.manufacturers.manufacturer?.id}
          }
        }
      }
    )
  }
`;
```

Die mit \$ gekennzeichneten Klammern, stellen die Werte dar, welche durch einen solchen Aufruf aus dem Interface extrahiert werden. `JSON.stringify` wird dabei dazu verwendet, um einen String zu versenden.

4.3 Dokumentation der Ergebnisse

Das Ergebnis meines 8-Wöchigen Praktikumsprojekt besteht aus einem auf die Kernfunktionen beschränktes Lagerverwaltungssystem. Dieses lässt sich sowohl auf einem Großen, als auch auf einem kleineren Display Problemlos bedienen und bietet eine Effektive und damit Zeitsparende UI/UX.

4.3.1 Funktionsumfang

Das System bietet folgende zentrale Funktionalitäten:

- **Artikelverwaltung:** Umfassende Darstellung aller Artikel mit detaillierten Informationen, effektive Suchfunktion sowie die Möglichkeit, neue Artikel hinzuzufügen und bestehende zu bearbeiten
- **Lagerplatzverwaltung:** Flexible Zuweisung von Artikeln zu beliebigen Lagerplätzen

- **Herstellerverwaltung:** Abruf und Anpassung von Herstellerinformationen
- **Lagerstättenverwaltung:** Übersichtliche Darstellung der Lagerstätten mit Anzeige der enthaltenen Artikelanzahl
- **Angebotsverwaltung:** Detaillierte Abfrage von Angebotsdetails
- **Sicherheit:** Zugriffsbeschränkung durch ein Login-System, das alle Funktionen erst nach erfolgreicher Authentifizierung freischaltet.

4.3.2 Technische Umsetzung

Die Komponenten sind durch die Nutzung des Vue-Frameworks einfach austauschbar und bieten große Flexibilität, was eine gute Erweiterbarkeit des Projekts mit sich zieht. Auch die Auslagerung von komplexeren TypeScript Funktionen unterstützt diese Eigenschaft.

Die Nutzung von schlichter und dunkler Farbe, gepaart mit herausstechenden Farben für wichtige Texte/Elemente, bietet ein modernes und ansprechendes Layout, welches die wichtigsten Informationen optimal herausstellt.

4.4 Diskussion der Ergebnisse

Das Ergebnis war gemessen an den festgelegten Erwartungshorizont sehr gut und ich hatte sehr viel Spaß dabei dieses Projekt zu diesem Endergebnis zu bringen. Alle Anfangs festgelegten Kernfunktionen, welche zum Abschluss des Projekts funktionieren sollten, konnten umgesetzt werden. Allerdings nahmen Anfangs einfache Funktionen schnell ungeahnte Maße an, welche in der Definition der Zielfunktionen nicht berücksichtigt wurden und dementsprechend nicht gewertet werden können.

Ein Beispiel dafür wäre das Einpflegen von Artikeln auf Lagerplätzen. Schien es anfangs doch so einfach zu klingen kamen plötzlich Fragen dazu auf, ob mehrere Artikel auf einem Lagerplatz lagern können oder auch ob die Maße und das Gewicht der Artikelmenge berücksichtigt werden müssen. Eine weiteren Anfangs nicht beachtete Eigenschaft war bspw. Auch, dass nicht die "Artikelanzahl" als Ganzes angepasst werden kann, da der Artikel ja möglicherweise auf mehreren Lagerplätzen liegt und somit nicht klar ist von welchem Lagerplatz die eingebende Menge nun abgezogen werden müsste.

Solche im Verlauf des Projekts auftretenden Fragen erforderten oft eine Umplanung, weshalb auch die Ziele laufend hätten angepasst werden müssen.

Die Verbindung zum Backend über GraphQL bereitete anfangs einige Verständnisschwierigkeiten, stellte sich jedoch schon nach kurzer Zeit als sehr nützlich heraus und war bald gut zu verstehen. GraphQL stellte die Abfrage von Expliziten Daten bereit und sorgte für effizientere Fetch-Abfragen, da nur die benötigten Daten geladen werden können und dies die Ladezeit massiv verringerte.

Insgesamt konnte ich das Projekt erfolgreich abschließen. Mein Praktikumsbetrieb zeigte sich sehr zufrieden mit meiner Arbeit und bot mir im Anschluss an das Praktikum einen Minijob zur Weiterentwicklung der Anwendung an. Dieses Angebot werte ich als wertvolle Bestätigung meiner Leistung.

Müsste ich das Projekt in der Form zu jetzigem Zeitpunkt noch einmal Umsetzen, würde ich mehr Zeit in die Planung stecken und Aufkommenden Herausforderungen vorbereiteter gegenüberreten, wodurch ich wiederum in der Umsetzung einiges an Zeit sparen könnte. Auch würde ich weniger Zeit in den Design Prozess legen bzw. vermehrt das Bootstrap-Framework einsetzen, um mich mehr auf die Funktionsweise zu konzentrieren. Bereits im Voraus wurde mir dies von meinen Praktikumsbetreuern geraten, um bei Zeitknappheit trotzdem ein funktionierendes Projekt vorweisen zu können.

Trotz alledem bin ich sehr zufrieden mit dem Endergebnis, da die Lagerverwaltungssoftware nach Abschluss des Praktikums bereits in ihrer initialen Zielsetzung einsetzbar ist. Weitere Funktionen wie das automatische auslösen von Bestellungen oder dem erstellen von detaillierten Monatsberichten, hätten den Rahmen gesprengt. Meine entwickelte Basis bietet allerdings eine gute Erweiterbarkeit um auch solche Funktionsweisen noch im Nachhinein gut implementieren zu können.

5 Tagesberichte

Mo., 10.02.2025:

Der erste Arbeitstag bestand ausfolgenden Aufgaben:

- Führung & Sicherheitseinweisung durch den Betrieb
- Aufbau meines Arbeitsplatzes für die folgenden Wochen
- Einrichten des Windows PCs und hinzufügen meines Benutzeraccounts zu der Windows Server Domäne
- Installieren der benötigten Software (JetBrains, Node.js, Bootstrap, Vue.js, Typescript, Chrome)
- Anmelden in Firmeninternen Systemen (Mail/Zeitplaner, Git)
- Einarbeiten in die installierte Software/Frameworks
- Aufbau eines Arbeitsplatzes für einen Mitarbeiter

Di., 11.02.2025:

Nachdem der erste Arbeitstag mehr aus Einrichtung und Vertrautheit mit der Software machen verstrich, begann ich am zweiten Tag mit meinem Projekt. Demnach fielen heute folgende Schritte an:

- Installieren und aufsetzen eines Vue.js Projekts in JetBrains mit benötigten nebenanlagen (Bootstrap & Typescript)
- Erstellen einer ausgelagerten "Header.vue"-Datei, welche mithilfe der vue.js Struktur in die Hauptstruktur "App.vue" geladen wurde
 - Grobe Skizzierung und Planung der navbar (mobile-first) und zusammensuchen der benötigten Icons
 - Platzierung der Elemente fixiert am oberen Bildschirmrand
- Einrichten einer Typescript Fetch-Funktion um auf bereits bestehende Serverstrukturen bzw. deren Informationen zugreifen zu können
 - Recherchieren dieser Funktion
 - Verständnissgewinnung durch die Nutzung von Nightingale REST Client
 - Implementierung
- Fehlerbeseitigung
 - Nach Neustart des vue.js Servers konnte dieser nicht erneut gestartet werden
 - Ausgelagerte "Stil.scss" Datei wurde plötzlich nichtmehr gefunden
 - Fehler beseitigt: vue.js wurde versucht aus Falschem Dateipfad zu starten
 - Fehler korrigiert
- Exkurs: git branch Nutzung

- Erneutes schreiben der durch Fehlerbehebung benötigte git Rollup gelöschten Type-script Fetch-Funktion
 - Problem: Cross-Origin Ressource Sharing (CORS) verhindert Zugriff, da der Client serverseitig im Header nicht zum Zugriff freigegeben wurde
 - Ausprobieren möglicher Lösungsansätze:
 - <https://medium.com/geekculture/bypassing-cors-with-a-google-chrome-extension-7f95fd953612>

Mi., 12.02.2025:

Nach gestrigen Schwierigkeiten mit dem Cross-Origin Ressource Sharing (CORS) Protokoll ging es heute daran diese zu beheben und den Fetch in der Konsole (später auch als Tabelle) darzustellen/auszugeben. Zudem lagerte ich anfallende SVGs in eine .scss Bootstrap Datei aus. Dies führte ich wie folgt durch:

- Einpflegen der benötigten CORS-Freigabe im Header (serverseitig)
- Fetch-Befehl Client seitig korrekt anpassen (URL ändern)
- Ausgabe mithilfe von "stringify" in der Konsole
- Ausgabe des Arrays in dem Webbrowser
- SCSS
 - Auslagern der SVG-Icons in eine separate .scss Datei (Bessere Übersichtlichkeit)
- Teilnahme an einem Digitalen Firmeninternen Meetings

Do., 13.02.2025:

Nach gestrigem Erfolg galt es nun die Daten geordnet in einer Tabelle darzustellen und die nötigen Suchfunktionen zu implementieren. Des weiteren vollendete ich heute auch die Option der Dateneinpflegung in die Datenbank (Ebenfalls mithilfe von Fetch()). Es folgt eine Detailreiche Auflistung:

- Beheben von stilistischen Fehlern (Wrapper geht über den Rand hinaus usw.)
- Darstellen der Daten in einer mithilfe von Bootstrap erstellten Tabelle
- Ersetzen der Hardcodierten Farben mithilfe von .scss Variablen
- Hinzufügen und optimieren einer Suchfunktion mithilfe von Bootstrap und TypeScript
- Option zum Hinzufügen von Daten in das bestehende Backend (Link via RouterLink)
- Optimierte Darstellung der "addData" Seite

Fr., 14.02.2025:

Heute war es so weit die Login-Page zu erstellen, um nicht dazu berechtigten Personen den Zugang zum Programm zu verweigern. Da das Backend mit dem Windows AD verknüpft ist, waren bereits alle Firmenmitglieder vorhanden. Nun galt es Benutzernamen und Passwort abzufragen. Dazu war folgendes an diesem Tag zu tun:

- Erstellen eines Login-Formulars (Login-Page)
- Auslesen der Form-Input Feldern und übergeben dieser an den Server (Weiterhin mit Fetch)
- Prüfen des Rückgabewerts auf "TRUE" -> (Die Anmeldung war erfolgreich) "FALSE" -> Falscher Benutzername oder Passwort

Mo., 17.02.2025:

An diesem Montag ging es um die Optimierung meines Vue-Codes, welcher nicht optimal angelegt war. Ich verschob beispielsweise meine Ansichten und definierte diese für jede Website in meiner "index.ts". Des weiteren erstellte ich das Design für die Seite "manufacturer-Table.vue" und gab dort die ausgelesenen Daten der Hersteller aus.

Di., 18.02.2025:

Mein heutiger Tag bestand aus folgenden Dingen

- Code review
- Weiterarbeit an meinem Projekt
 - Login - auth_ad to string -> Store string in variable and send back in header to the server to gain customized web access
 - Destruct session X-api-key
 - Change App.vue display (restricted view) to index.ts view control
 - Set loginForm.vue as primary page | If not filled -> no access to the following pages
 - "manufacturerTable" is displayed on "/home"
 - Display navbar(header) on "/manufacturerTable"

Mi., 19.02.2025:

Nachdem mein Projekt in den letzten Tagen flüssig und ohne Probleme weiterentwickelt werden konnte, war heute ein sehr stockender Tag. Erfolgreich implementiert wurde folgendes:

- Einbinden von Bildern mithilfe von der Base64 vom Server übergebenen Base64 Kodierung

Implementiert und verworfen wurde folgendes:

- Umbau des Vue.js Projekts – von Einschränken der Komponenten auf verschiedensten Seiten zu erlauben von bestimmten Komponenten auf definierten Seiten (in der main.ts)
 - Die Einschränkungspolitik ist nicht das von Vue dazu konzipierte Verfahren -> Zudem werden Codes so unglaublich lang und unübersichtlich
- Umbau der Flex Anordnung zu einem Grid-Layout
- Aufruf einer Typescript-Funktion in Komponente A von Komponente B aus
- **Verwerfen dieser Änderungen mithilfe von den stetig gepushten git commits (Rollback)**

Mi., 19.02.2025:

Nach gestrigem Misserfolg ging es heute daran die verworfenen Ideen in etwas anderer Form erneut zu implementieren.

- Führung durch die Serverräume / Siemens Automatisierung
- Durchlaufen eines Typischen Prozesses, wie langsame Computergeschwindigkeiten zustande kommen und was unternommen werden kann mit Betreuer Robin Christes
- Theoretische Einführung in Firewalls und Gespräch mit Timo Richter über Embeded Systems (GNU CC Compiler und Assembler)
- Abruf der "History" durch Fetch-Anfragen
 - Umwandlung des Unix-Timestamp zu DD/MM/YYYY & HH:MM:SS
 - Darstellung dieser in einer gesonderten Seite
- Erneuter Umbau der Vue-Struktur zu: "Erlauben bestimmter Komponenten"
- Erneute Implementierung des Grid-Layouts -> Diesmal ausgelagert und für jede "View" gesondert anpassbar

Do., 20.02.2025:

Heute begann der Tag mit einem Standardmäßigen git, welcher mich nun leider in meine neuste Version katapultierte, welche alles gestern implementierte natürlich nicht beinhaltete, da ich diese Änderung auf einem älteren Stand vorgenommen habe. Hier folglich, dass sich mein gestriger Tag im Wesentlichen wiederholte.

Fr., 21.02.2025:

An diesem Freitag hatte ich einige Aufschlussreiche Gespräche mit Mitarbeitern über die Themen Embeded Systems und V-LANs. Des Weiteren konnte ich mich gestalterisch betätigen,

indem ich einige Buchstaben und Zahlen aus einer neuen Schriftart, welche für den Firmenauftritt genutzt werden sollen, neu designte, was einen besseren Lesefluss ermöglicht. Des Weiteren arbeitete ich weiter an meine Website.

Mo., 24.02.2025:

Heute verlief der Tag sehr erfolgreich. Ich arbeitete an meinem Projekt weiter und lagerte einige Funktionen in externe Dateien aus, um den Code übersichtlicher zu gestalten. Auch wurden einige Doppelungen entfernt. Des Weiteren wurden folgende Weiterentwicklungen fertiggestellt:

- Animieren der "productView.vue" Komponente bei Hover über jeweilige Spalten in der Komponente fetchAPI.vue
- Auslesen der Spalte mithilfe des übergebenen Spaltenindex
- Darstellen der zu der jeweiligen Spalte gehörigen Daten in der "productView.vue" Komponente
- Auslesen und darstellen der Anzahl an Artikeln
- Anpassen der in Base64 dargestellten Bildern

Di., 24.02.2025:

Auch diesen Dienstag ging es daran das Projekt voranzutreiben. Auf Wunsch meiner Betreuer wurde die Komponentenanzahl etwas verändert. Dies war dank des genutzten Grid-Layouts kein großer Akt. Des Weiteren konnte die Optimierung des Fetch-Request, welcher zum Laden der Website nun vorerst nur die nötigsten Daten lädt und nähere Informationen in Zukunft nachladen, die zu ladenden Datenmengen von 5,2mb auf 4,7kb verringern. Dies garantiert schnellere Ladezeiten der Website. Auch werden den Artikeln nun nichtmehr manuelle Nummerierungen zugewiesen. Diese werden nun mit einer Sortierfunktion aus der "id" der Artikel gelistet.

Mi., 24.02.2025:

Die sogenannte Lazy-Loading Funktion konnte heute erfolgreich umgesetzt werden. Dadurch werden jetzt nur die nötigsten Daten gelesen und weiteres bei gebrauch nachgeladen. Besonders die in Base64 codierten Bilder, welche nun Anfangs nicht in der Abfrage enthalten sind verschnellern das Laden um ein Vielfaches.

Des weiteren konnte das Design der Homepage mit vorrausgehender, zuhause erstellter Planung, um einiges aufgewertet werden. Auch wurde die "productView"-Komponente dahingehend angepasst, sodass sich nun zwei Ansichten "Display" und "Edit" ergeben, welche durch das Anpassen eines Variablenwerts geschaltet werden können. Auch lässt sich die angesprochene Komponente mit Klick auf ein Icon fließend vergrößern.

Do., 24.02.2025:

Die "productView"-Komponente konnte heute mit weiteren Informationen gefüllt werden. Auch wird der Produktbestand nun farblich und mit Anmerkung in "Bedenklich", "Sofortig Nachfüllen" und "Gut" sortiert angezeigt. Auch konnte die zweite Ansicht, in welcher sich das aktuell ausgewählte Produkt editieren (und die dementsprechenden Änderungen an den Server weitergeben) lassen fertig gestellt werden. Auch wird das Produktbild nun ebenfalls als Datei dargestellt und lässt sich mit einem Klick herunterladen.

Fr., 25.02.2025:

Zum Abschluss dieser Woche galt es meine UI an das Farbschema des von Richter Elektronik genutzten Internetauftritt anzupassen. Dazu erschuf ich mithilfe von "Adobe Farbpalette" ich eine .scss Farbpalette, welche ich auf meine UI anhand. Des Weiteren machte ich mich daran die UI auf mobil Telefon Größe anzupassen, da die Website perspektivisch auch auf Zebra-Scannern laufen soll.

Mo., 03.03.2025:

Zum Rosenmontag begann ich mit einem kurzen eingeschobenen Projekt. Eine Firewall des Herstellers Thomas Krenn muss für den Backup-Server der Richter Elektronik GmbH eingerichtet werden. Dazu muss folgendes getan werden:

- Image von OPNSense herunterladen <https://opnsense.org/download/>
- Mit Rufus auf einen USB-Stick schreiben
- Gerät auspacken und an dem Bastelplatz einstecken
- Installieren und updaten
- Als Endgerät hinter dem Server wird ein Laptop genutzt
- Der zukünftige Aufbau ist wie folgt gedacht
 - Sicherungsserver (aktuell der Laptop) <-> FW <-> Richter-Netz
 - Um an den Sicherungsserver zu kommen, sollte aus dem Richter-Netz ein VPN an den Sicherungsserver möglich sein
 - <https://docs.opnsense.org/manual/vpnet.html#new-vpn-openvpn-instances>
 - Hierzu wird ein SSL-VPN genutzt
 - Verbindung mit dem Richter-Netzwerk herstellen
 - Sinnvoll wären hier auch noch Punkte wie QoS, bzw. „bandwith throttling“, um die Netzwerkleistung des Sicherungsservers zu unseren Hyper-V-Hosts zu beschränken

Di., 04.03.2025:

Da die Einrichtung der Firewall gestern nur zum Teil zu Ende gebracht wurde. Ging es heute an den restlichen Teil. Der Einrichtung des OpenVPN. Dazu nahm ich mir die Dokumentation von OPNSense zur Hand und begann mit der Einrichtung. Nach einiger Zeit stieß ich auf einige Probleme, welche selbst mithilfe von Herrn Christes und mit einem Zeitaufwand von ca. 7 Stunden nicht gelöst werden konnten und bis zum Abschluss des Tages unvollendet blieben. Weitere Vorhaben (Idee: Nutzung eines anderen VPN-Anbieters bspw. Wireguard) wurden auf den nächsten Tag verschoben.

Mi., 05.03.2025:

Heute begann ich mit der Einrichtung eines Wireguard VPN's auf meiner Firewall. Dies war ohne große Probleme Möglich, bis es an die Einrichtung des Clients ging. Hier hörte die Dokumentation abrupt auf und wir entschieden uns an meinem eigentlichen Projekt weiterzuarbeiten und den Server einzurichten, wenn genug Zeit dafür ist. Zu den Komplikationen bei der Firewall (OPNSense) kam es, da die UI vor kurzer Zeit vollständig umgebaut wurde und somit noch keine neuen Anleitungen/Dokumentationen zu dem neuen System existieren.

Do., 06.03.2025:

Heute bestand mein Tag daraus weitere Fetch-Abfragen hinzuzufügen und damit mein Projekt voranzutreiben. Zudem bekam ich ein Laptop an die Hand, welchen ich einrichtete und welcher in Verbindung mit einer Docking-Station nun mein Hauptrechner darstellen wird, welchen ich auch zuhause für in Zukunft fürs Homeoffice usw. nutzen kann. Gegen Mittag begann mein Betreuer Herr Christes damit einen Webserver für die Seite zu erstellen. Da es dort einige Probleme gab und ich zeitweise kein Zugriff auf die GraphQL-Schnittstelle hatte, welche essenziell für meine Abfragen sind, begann ich später damit ein Responsives Design für meine Website zu schaffen, um die Website sobald der Webserver steht bereits auf den Scannern (Zebra TC21) zu testen.

Fr., 06.03.2025:

Zum Abschluss der Woche konnte die "Stockpile" Website erfolgreich auf einen Webserver geladen werden und kann demnach nun intern von jedem Gerät aufgerufen werden. Des Weiteren konnte das Design der "/addEntry"-Seite überarbeitet und auf dem Zebra Scanner geladen/getestet werden. Auch konnte ich somit mein Input für den Barcode-Scanner erfolgreich testen. Auch konnten wir zwei Probleme (Druckerproblem und Zebra-Scanner Ladefunktion) erfolgreich beheben bzw. Reparaturen in Auftrag geben. Auch wurde mir das Thema "**Regular Expressions**" in der Programmierung nähergebracht.

Mo., 10.03.2025:

Die Woche begann mit dem weiteren Ausbau der Webapp auf eine an das "Zebra" angepasste Version. Auch konnte eine Komponente geschaffen werden, welche in einem Dropdown-Menü mögliche Lagermöglichkeiten für neue Produkte aufzeigt. Auch kann dazu bereits ein passender Produktname eingegeben werden und mithilfe einer Mutation serverseitig gespeichert werden. (Noch nicht voll funktionsfähig)

Di., 10.03.2025:

Es folgte auch heute der Ausbau der Komponente mithilfe es möglich sein soll, neue Produkte mit dazu passendem Lagerplatz hinzuzufügen. Dazu wurde die Reihenfolge geändert - *Statt Eingabe eines Lagerplatzes und einem dazu passenden Artikel, soll einem Artikel (Anzahl) ein passender Lagerplatz angezeigt werden* - und ein Dropdown-Menü hinzugefügt, welches während Eingabe eines Artikelnamens Vorschläge / mögliche Treffer listet und bei Auswahl alle dazu passenden weiteren Informationen lädt. Diese sind standardmäßig nicht anpassbar, sondern nur mit einem Klick auf den "Edit"-Button möglich.

Mi., 11.03.2025:

Auf einige erfolgreiche Tagen erfolgt ein sich sehr in die Länge ziehende Tag. Bei der Implementierung der Anzeige von Lagerplatzvorschlägen zu dem jeweiligen Artikel, stieß ich auf einige Probleme aufgrund von einem nicht aufgeräumten Code. Zum Ende des Tages beschloss ich den Funktionsteil (TypeScript) bis auf einige gestalterischen Funktionen vollständig zu entfernen, um mich morgen mit der Neucodierung dieser beschäftigen kann.

Do., 12.03.2025:

Mit einem fast vollständigem Blanko-Script Part startete ich heute in die Entwicklung der Lagerplatzvorschläge. Dies funktionierte ohne weitere Probleme und so konnte der Code um ca. 150 Zeilen minimiert werden. So kann eine bessere Übersichtlichkeit gewährleistet werden.

Fr., 13.03.2025:

Der Freitag wurde mit der Funktion des Ändern des Produktstocks (Produktanzahl im Lager) abgerundet. So konnte nun bereits 1 von 3 Zielen, welche Anfangs mit dem Betrieb entwickelt wurden umgesetzt werden. Artikel lassen sich mithilfe einer Suchleiste und Vorschlagsliste auswählen, die passenden Infos dazu anzeigen und die Artikel Anzahl ändern. Des Weiteren durfte ich den Firmenchef Tobias Richter kennenlernen, welcher mir einen Minijob anbot.

Mo., 17.03.2025:

Die Woche begann damit, ein Button hinzuzufügen, welcher die aktuelle Anzahl eines Artikels im Lager darstellt und diesen Wert mit einer $\cdot -1$ Rechnung zu invertieren und dies an den Server zurückzuschicken. Herr Christes setzte sich im Backend daran eine Funktion zu schreiben, welche prüft, ob die Artikelanzahl ≤ 0 ist, sollte dies der Fall sein, sollte der Artikel aus dem Lagerbestand gelöscht werden. So ist es schlussendlich möglich Artikel mithilfe der Suchleiste auszuwählen und diese vollständig aus dem Lagerbestand zu löschen.

Di., 18.03.2025:

Heute beendete ich die Funktion mit, welcher sich ein Artikel aus dem Lagerbestand löschen lässt, und fügte eine Warnung hinzu, welche die Abfrage stellt, ob die Komponente tatsächlich aus dem Lagerbestand gelöscht werden sollte. Gegen Nachmittag hin stellten sich einige Probleme heraus. Plötzlich ließen sich keine Artikel mehr hinzufügen, löschen oder gar ändern. Das Problem konnte ich schlussendlich nicht lösen und verschob weitere Vorhaben auf den morgigen Tag.

Mi., 19.03.2025:

Für den heutigen Tag war der Praktikumsbesuch meines Schulischen Betreuers Herr Maluck angesetzt, weshalb ich mich gleich früh am Morgen daran begab die gestrigen Probleme zu lösen, was nach kurzer Zeit auch möglich war. Dann war es auch schon 10:00Uhr und ich begab mich auf den Weg in den Besprechungsraum, in welchem ich nötige Vorkehrungen für die spätere Kurzpräsentation traf. Der Besuch verlief reibungslos und nach kurzer Präsentation meines Projekts und meines Betriebes, gab es noch eine schnelle Führung durch den Betrieb. Den Nachmittag nutzte mein Betreuer Herr Christes mich in die Software "NinjaOne" einzuführen, da ich diese für meine auf das Praktikum folgende Tätigkeit bei Richter Elektronik (in Form eines Minijobs) benötigen werde. Ebenfalls konnten wir erfolgreich einen PXE-Server aufsetzen.

Do., 20.03.2025:

Heute begann ich mit der Codierung der Seite, mit welcher sich bereits erstellte Artikel einem Lagerplatz zuweisen lassen. Die Darstellung beendete ich soweit und auch die Abfrage der Artikel (mit zugehöriger ID) war schnell gemacht. Ich hielt mich allerdings bis zum Ende des Tages damit auf, alle existierenden Stockpiles abzurufen und zur Auswahl zu stellen. Schließlich stellte sich heraus, dass dieses Problem serverseitig seinen Ursprung findet, welchen wir morgen weiter versuchen werden zu beheben.

Fr., 21.03.2025:

Mit Ende der 6-ten Woche war es mir heute möglich, alle existenten Stockpiles abzurufen und in einem Dropdown auszuwählen. Auch machte ich mich bereits an die Mutation, welche die Ausgewählten Artikel mit zugewiesenem Lagerplatz zurückschicken sollte. Die meiste Zeit hat mich das Design gekostet.

Mo., 24.03.2025:

Die vorletzte Woche begann mit der Erweiterung der Weboberfläche, mit welcher sich Artikel bereits existierenden Lagerorten zuordnen lassen. Ich implementierte die Filterfunktion der Suchleiste und erstellte weitere Fetch-Abfragen, welche alle existierende Stocknamen und Regale/Fächer abbildete. Dann implementierte ich noch eine Verknüpfung dieser Lagernamen zu den Regalen/Fächern um nur die in dem jeweilig existierenden Lager befindlichen Fächer/Regale abzubilden.

Di., 25.03.2025:

Den Dienstag verbrachte ich mit der erfolgreichen restlichen Implementierung der Funktion, mit welcher sich existierende Artikel, Lagerstätten zuweisen lassen. Dazu musste ich meine existierenden Stockpiles getrennt voneinander Abfragen. Erst alle existierenden Lagernamen, dann die dazu existierenden Regalnamen und wiederum dazu existierende Fächer. Regal- und Fachnamen müssen dann wieder zusammengefügt Abgefragt werden und dazu die passende ID ausgegeben werden. Diese kann dann schlussendlich mit der ausgewählten Artikel ID und dem gewünschten *"change"* zurückgegeben werden.

Mi., 26.03.2025:

Und schon wieder ist eine halbe Woche rum. Der heutige Tag bestand größtenteils aus dem Beheben von Problemen. Erst machte ich mich an die korrekte Darstellung meiner zur Auswahl stehenden Fächer, welche bei Auswahl seltsamerweise unsichtbar wurden. Es folgte die Suche nach einem Problemverursacher, welcher scheinbar parallel versuchte, die Artikel zu laden, welches zu Überlastungen und Kollisionen dieser Anfrage führte. Daraus resultierte, dass die Artikel nur manchmal korrekt geladen wurden. (Nur wenn alle 3 Anfragen korrekt bearbeitet wurden) Durch auskommentieren untergegangener und unbenutzter Funktionsaufrufen konnte auch dieses Problem behoben werden. Desweiteren konnte ich die Hersteller Tabelle für die Mobileansicht anpassen.

Do., 27.03.2025:

Heute machte ich mich an die Edit-Ansicht, mit welcher sich neue Hersteller Hinzufügen lassen können. Dies setzte ich sowohl für die Mobile-Ansicht als auch für die auf einem Computer um.

Fr., 28.03.2025:

Den Abschluss der Woche bildete ein Überprüfen des bisherigen Standes. Funktioniert alles so wie es soll? Und wenn nein, wo gibt es noch Probleme? Dabei taten sich einige Probleme im Bereich "Stockpile Anzahl ändern" und "Artikel umlagern" auf, welche schlussendlich nicht ganz behoben werden konnten und auf Montag verschoben wurden.

Mo., 31.03.2025:

Die letzte Woche begann mit dem Beheben von den am Freitag festgestellten Problemen. Diese Aufgabe lag allerdings schlussendlich bei meinem Betreuer Robin Christes, welcher das Problem schnell lösen konnte. Auch machte ich mich an die Abfrage weiterer Artikeldaten. Des Weiteren entwickelte ich ein kleines Tool in Python, welches automatisch Registriert, wenn ein Stick angeschlossen wird und dann darauf einen Virenskan namens ClamAV ausführt, welcher schlussendlich einen Status "Ok" oder "Gefährlich" zurückgibt. Dieses konnte ich allerdings noch nicht testen, da es für einen Linux Rechner konzipiert wurde. Auch soll dieses im späteren Verlauf über eine GraphQL Schnittstelle aufrufbar sein und somit den Status etwas klarer und einfacher darstellen können.

Di., 01.04.2025:

Der heutige Tag bestand aus der Erweiterung der ProductView-Edit Ansicht mit welcher sich Artikeldaten wie bspw. Lagerplätze und Anzahl verändern lassen können. Dies ist auf Mobilien Geräten bereits möglich und soll nun auch auf größeren Geräten möglich sein. Des Weiteren nahm ich mithilfe meines Betreuers einen Windows 11 Pro Rechner in Betrieb und installierte die UPS-Software auf dem Gerät eines Mitarbeiters im Vertrieb.

Mi., 02.04.2025:

Heute konnte der Windows Rechner, welchen ich gestern soweit auf Windows 11 installiert habe fertig gestellt werden. Um mir den Ablauf und die Ordner zu merken, schrieb ich zudem eine kleine Dokumentation dazu. Auch traten Probleme bei zwei Mitarbeitern auf, welche nicht Remote behoben werden konnten, weshalb sich mein Betreuer dorthin auf den Weg machte, wobei ich ihn Begleitete, um einen ersten Eindruck für meine spätere Minijob Tätigkeit zu erlangen. Zudem konnte die ProductView Page weitestgehend fertig gestellt werden, wodurch nun nur noch die jeweiligen Inputs zu der Fetch-Mutation verknüpft werden müssen. Mit dieser Übersicht ließen sich dann schlussendlich folgende Änderungen an einem Artikel vornehmen:

1. Artikelnamen verändern
2. Artikelbeschreibung verändern
3. Artikelhersteller verändern
4. Stückzahl an einem Lagerplatz verändern
5. Weiteren Lagerplatz wählen
6. Produktbild ändern
7. Zusätzliche Dateien hochladen/Löschen

Eine weitere noch umzusetzende Funktion wäre:

1. Aus bestehendem Lagerplatz in einen anderen Lagerplatz umlagern

Do., 03.04.2025:

Heute durften wir einen neuen, von Extern eingekauften, Mitarbeiter begrüßen, welcher in nächster Zeit mit mir und Herr Christes den IT-Support der Richter Elektronik GmbH bilden wird. Diesen gaben wir eine kleine Führung durch die Firma, bei welcher Herr Christes uns beide nebenbei als neue Mitglieder des Supports bei den Angestellten vorstellte. Des Weiteren überprüfte ich mein Projekt auf Fehler und begab mich daran diese noch zu beheben.

Fr., 04.04.2025:

Auch heute das stand beheben von Fehlern auf dem Plan. Dazu musste ich unter anderem eine Mutation noch einmal vollkommen neu schreiben, welche die Funktionsweise zum Hinzufügen von Artikeln bildet. Auch machte ich mich daran, meine Facharbeit zu erweitern.

6 Eigenständigkeitserklärung

Mit meiner Unterschrift erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst habe und keine anderen als die angegebenen Quellen benutzt habe. Alle Stellen dieser Arbeit, die dem Wortlaut, dem Sinn oder der Argumentation nach anderen Werken und/oder mithilfe von Werkzeugen entnommen sind (Internet, Zeitschriften, Bücher, KI-Chatbots), habe ich unter Angabe der Quellen vollständig kenntlich gemacht.

Olsberg, den _____
(Datum)

(Unterschrift des/r Praktikanten/in)

7 Praktikumsbescheinigung



Schmallenberg, 04.04.2025

Praktikumszeugnis

Herr Paul Dobrick, geboren am 15.04.2006 in Lennestadt, hat vom 10.02.2025 bis 04.04.2025 ein Praktikum in der Abteilung IT in unserem Unternehmen absolviert. Herr Dobrick hatte insbesondere folgende Aufgaben

- Entwicklung eines Web-Frontends mit TypeScript
- Einrichtung von IT-Systemen an Arbeitsplätzen
- Konfiguration von Netzwerkgeräten

Die Richter Elektronik GmbH produziert und vertreibt unter der Marke Richter Leiterplatten hochwertige Leiterplatten und Multilayer für weltweite Anwendungen. Das 1978 gegründete Familienunternehmen gehört zu den Hightech-Unternehmen in der Region Schmallenberg und liefert Leiterplatten für die E-Mobilität, Kommunikationstechnik, Medizintechnik, automobiler Prototypen, die Leuchtenindustrie und für viele weitere innovative Produkte. Überall in der Welt arbeiten Elektronik-Systeme mit Leiterplatten, die in Schmallenberg produziert wurden.

Um den reibungslosen Betrieb unserer komplexen Fertigung mit einer Vielzahl an unterschiedlichen Prozessen jederzeit aufrecht zu erhalten und die Flexibilität und Qualität kontinuierlich zu verbessern, verfügt das Unternehmen zudem über umfangreiche hausinterne Fähigkeiten im Bereich IT und Software-Entwicklung.

Herr Dobrick verfügt über gute Vorkenntnisse, die er zur Bewältigung seiner Aufgaben stets sehr sicher und erfolgreich einsetzte. Er hat eine rasche Auffassungsgabe und konnte auftretende Probleme schnell und zutreffend lösen. Er arbeitete stets selbständig, sicher und sehr motiviert. Seine Arbeitsergebnisse waren stets von sehr guter Qualität und haben unsere Erwartungen übertroffen.

Er integrierte sich sehr gut in unsere Teamstruktur. Herr Dobrick zeichnete sich durch seine sehr vertrauenswürdige, hilfsbereite und loyale Art aus. Von Vorgesetzten und Kollegen wurde er jederzeit geschätzt.

Wir danken Herrn Dobrick für seine wertvolle Mitarbeit und wünschen ihm für seine Zukunft und seine weitere schulische Ausbildung alles Gute und auch weiterhin viel Erfolg.

Wir freuen uns sehr darüber, dass uns Herr Dobrick zukünftig im Rahmen eines Minijobs weiter für uns tätig sein wird.



Tobias Richter
Geschäftsführer



i.A. Robin Christes
IT Operations & Software Engineer

Richter Elektronik GmbH | Hünegräben 6 | 57392 Schmallenberg | Deutschland
richter-leiterplatten.de | +49 2972 9796-0 | Geschäftsführer: Tobias Richter B.Sc.
AG Arnsberg HRB 2330 | USt-IdNr.: DE811377291 | Steuer-Nr.: 533457960055
Volksbank Sauerland eG | IBAN: DE91 4606 2817 0014 3466 00 | BIC: GENODEM333

DIE ZUKUNFT SCHALTET RICHTER



Literaturverzeichnis

- [1] <https://docs.github.com/de/issues/tracking-your-work-with-issues/about-issues> (11.03.2025 - 17:32)
- [2] <https://bsh-ag.de/it-wissensdatenbank/framework/> (12.03.2025 – 20:47)
- [3] <https://html.spec.whatwg.org/multipage/introduction.html#html-vs-xhtml> (20.03.2025 – 19:33)
- [4] <https://de.ryte.com/wiki/Stylesheet/> (20.03.2025 – 19:46)
- [5] <https://www.redhat.com/de/topics/api/what-is-graphql#:~:text=Ein%20GraphQL%2DSchema%20definiert%2C%20welche,mit%20einer%20Funktion%20namens%20Resolver>. (27.04.2025 – 18:38)
- [5] <https://www.onelogin.com/de-de/learn/what-is-ldap#:~:text=Bei%20der%20LDAP%2DAuthentifizierung%20werden,erstelen%20und%20ihnen%20Berechtigungen%20zuweisen>. (28.04.2025 – 12:47)
- [6] <https://www.prosec-networks.com/blog/active-directory-einfach-erklart/> (28.04.2025 – 12:50)
- [7] <https://developer.mozilla.org/de/docs/Web/HTTP/Reference/Methods> (30.04.2025 - 13:42)
- [8] https://www.w3schools.com/tags/ref_httpmethods.asp (30.04.2025 - 13:40)
- [9] <https://entwickler.de/php/trennungsfreuden-001> (04.05.2025 - 11:42)
- [10] <https://getbootstrap.com/docs/4.3/getting-started/introduction/> (04.05.2025 – 17:21)

ⁱ Das Frontend: Der sichtbare Teil einer Anwendung, mit dem Nutzer interagieren – z. B. über Schaltflächen, Formulare oder Ansichten im Webbrowser

ⁱⁱ Die technische Gesamtstruktur eines Softwaresystems, die zeigt, wie verschiedene Komponenten (z. B. Benutzeroberfläche, Server, Datenbank) miteinander verbunden und organisiert sind.

ⁱⁱⁱ Eine Datenbank ist ein System zur strukturierten Speicherung, Verwaltung und Abfrage von Daten.

^{iv} Gitea ist eine leichte, selbst gehostete Git-Plattform zur Versionsverwaltung von Code, ähnlich wie GitHub oder GitLab. Sie bietet Funktionen wie Repository-Verwaltung, Issues, Pull Requests und Benutzerverwaltung – alles in einer einfach installierbaren Webanwendung.

^v Kombination von Programmiersprachen, Frameworks, Tools und Technologien, die zusammen verwendet werden, um eine Softwareanwendung zu entwickeln und zu betreiben.

^{vi} Schnittstellen, die es ermöglichen, Daten über das Internet mithilfe von HTTP-Anfragen (z. B. GET, POST, PUT, DELETE) zwischen Client und Server auszutauschen

^{vii} Pinia ist der offizielle State-Management-Store für Vue.js (ab Vue 3) und dient dazu, Anwendungsdaten zentral zu verwalten und zwischen Komponenten zu teilen.

- viii Ein Debounce-Mechanismus sorgt dafür, dass eine Funktion erst nach einer bestimmten Verzögerung ausgeführt wird, nachdem sie zuletzt aufgerufen wurde – und nur einmal, selbst wenn sie mehrmals schnell hintereinander ausgelöst wurde
- ix Grundprinzip der Softwareentwicklung: Unterschiedliche Verantwortlichkeiten innerhalb eines Programms sollen sauber voneinander getrennt werden.
- x Eine Datenabfrage, welche zur Abfrage von Daten einer Schnittstelle (In meinem Fall GraphQL genutzt wird)
- xi Vorgang läuft unabhängig und gleichzeitig mit anderen Abläufen